

List of experiments:

- 1. Build a responsive web application for shopping cart with registration, login, catalog and cart pages using CSS3 features, flex and grid.**
- 2. Make the above web application responsive web application using Bootstrap framework.**
- 3. Use JavaScript for doing client – side validation of the pages implemented in experiment 1 and experiment 2.**
- 4. Explore the features of ES6 like arrow functions, callbacks, promises, async/await. Implement an application for reading the weather information from openweathermap.org and display the information in the form of a graph on the web page.**
- 5. Develop a java stand alone application that connects with the database (Oracle / mySql) and perform the CRUD operation on the database tables.**
- 6. Create an xml for the bookstore. Validate the same using both DTD and XSD.**
- 7. Design a controller with servlet that provides the interaction with application developed in experiment 1 and the database created in experiment 5.**
- 8. Maintaining the transactional history of any user is very important. Explore the various session tracking mechanism (Cookies, HTTP Session)**
- 9. Create a custom server using http module and explore the other modules of Node JS like OS, path, event.**
- 10. Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman)**
- 11. For the above application create authorized end points using JWT (JSON Web Token).**
- 12. Create a react application for the student management system having registration, login, contact, about pages and implement routing to navigate through these pages.**
- 13. Create a service in react that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js**
- 14. Create a TODO application in react with necessary components and deploy it into GitHub.**

Experiment 1

Objective: Build a responsive web application for a shopping cart with registration, login, catalogue, and cart pages using CSS3 features, flex, and grid.

To implement this, we will create a shared CSS file (style.css) utilizing CSS Flexbox and Grid, along with four HTML files:

1. register.html (Registration Page)
2. login.html (Login Page)
3. catalog.html (Product Catalogue Page)
4. cart.html (Shopping Cart Page)

style.css (Common to all pages)

```
/* Reset and General Styles */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
  background-color: #f4f6f9;
  color: #333;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}

/* Header & Navigation (Flexbox) */
header {
  background-color: #1a252f;
  padding: 15px 5%;
}

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  flex-wrap: wrap;
}

.brand {
  color: #fff;
  font-size: 1.5rem;
  font-weight: bold;
  text-decoration: none;
}

.nav-links {
  display: flex;
  list-style: none;
  gap: 20px;
}
```

```
.nav-links a {
  color: #bdc3c7;
  text-decoration: none;
  font-size: 1rem;
  transition: color 0.3s;
}

.nav-links a:hover, .nav-links a.active {
  color: #fff;
}

/* Container */
.container {
  flex: 1;
  width: 90%;
  max-width: 1200px;
  margin: 30px auto;
}

/* Forms (Flexbox Alignment) */
.form-container {
  background-color: #fff;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  max-width: 450px;
  margin: 40px auto;
}

.form-container h2 {
  margin-bottom: 20px;
  color: #2c3e50;
  text-align: center;
}

.form-group {
  margin-bottom: 15px;
  display: flex;
  flex-direction: column;
}

.form-group label {
  margin-bottom: 5px;
  font-weight: 600;
}

.form-group input {
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
  font-size: 1rem;
}

.btn {
  background-color: #3498db;
  color: #fff;
  padding: 10px 15px;
  border: none;
  border-radius: 4px;
}
```

```
font-size: 1rem;
cursor: pointer;
width: 100%;
transition: background-color 0.3s;
}
```

```
.btn:hover{
  background-color: #2980b9;
}
```

```
.form-footer{
  margin-top: 15px;
  text-align: center;
  font-size: 0.9rem;
}
```

```
.form-footer a{
  color: #3498db;
  text-decoration: none;
}
```

```
/* Catalog Grid Layout */
```

```
.catalog-grid{
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 25px;
}
```

```
.product-card{
  background-color: #fff;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  padding: 15px;
  display: flex;
  flex-direction: column;
  justify-content: space-between;
}
```

```
.product-image{
  width: 100%;
  height: 180px;
  background-color: #ecf0f1;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 0.9rem;
  color: #7f8c8d;
  border-radius: 4px;
  margin-bottom: 15px;
}
```

```
.product-info h3{
  font-size: 1.2rem;
  margin-bottom: 8px;
  color: #2c3e50;
}
```

```
.product-info p{
  font-size: 0.9rem;
  color: #7f8c8d;
}
```

```
margin-bottom: 12px;
}

.product-price {
  font-size: 1.2rem;
  font-weight: bold;
  color: #e74c3c;
  margin-bottom: 15px;
}

/* Cart Page Layout (Flexbox & Grid) */
.cart-layout {
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 30px;
}

@media (max-width: 768px) {
  .cart-layout {
    grid-template-columns: 1fr;
  }
}

.cart-items {
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

.cart-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 1px solid #ecf0f1;
  padding: 15px 0;
}

.cart-item:last-child {
  border-bottom: none;
}

.cart-item-info {
  display: flex;
  gap: 15px;
  align-items: center;
}

.cart-item-img {
  width: 60px;
  height: 60px;
  background-color: #ecf0f1;
  border-radius: 4px;
}

.cart-item-details h4 {
  font-size: 1rem;
  color: #2c3e50;
}
```

```
.cart-item-details p {
  font-size: 0.85rem;
  color: #7f8c8d;
}

.cart-item-actions {
  display: flex;
  align-items: center;
  gap: 15px;
}

.quantity-input {
  width: 50px;
  padding: 5px;
  text-align: center;
}

.remove-btn {
  background-color: #e74c3c;
  color: white;
  border: none;
  padding: 5px 10px;
  border-radius: 4px;
  cursor: pointer;
}

.cart-summary {
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  height: fit-content;
}

.summary-row {
  display: flex;
  justify-content: space-between;
  margin-bottom: 15px;
  font-size: 1rem;
}

.summary-total {
  border-top: 1px solid #ecf0f1;
  padding-top: 15px;
  font-size: 1.2rem;
  font-weight: bold;
}

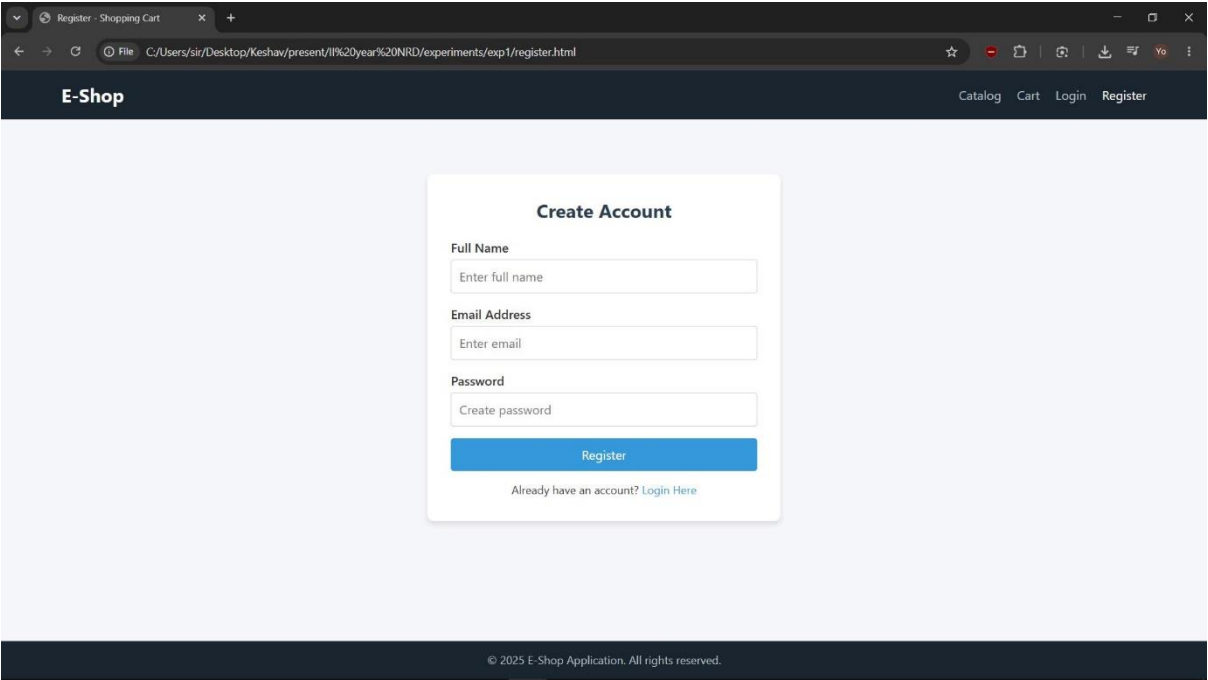
/* Footer */
footer {
  background-color: #1a252f;
  color: #bdc3c7;
  text-align: center;
  padding: 15px;
  margin-top: auto;
  font-size: 0.9rem;
}
```

1. register.html (Registration Page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Register - Shopping Cart</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <nav class="navbar">
      <a href="catalog.html" class="brand">E-Shop</a>
      <ul class="nav-links">
        <li><a href="catalog.html">Catalog</a></li>
        <li><a href="cart.html">Cart</a></li>
        <li><a href="login.html">Login</a></li>
        <li><a href="register.html" class="active">Register</a></li>
      </ul>
    </nav>
  </header>

  <div class="container">
    <div class="form-container">
      <h2>Create Account</h2>
      <form action="login.html" method="GET">
        <div class="form-group">
          <label for="username">Full Name</label>
          <input type="text" id="username" required placeholder="Enter full name">
        </div>
        <div class="form-group">
          <label for="email">Email Address</label>
          <input type="email" id="email" required placeholder="Enter email">
        </div>
        <div class="form-group">
          <label for="password">Password</label>
          <input type="password" id="password" required placeholder="Create password">
        </div>
        <button type="submit" class="btn">Register</button>
      </form>
      <div class="form-footer">
        Already have an account? <a href="login.html">Login Here</a>
      </div>
    </div>
  </div>

  <footer>
    &copy; 2025 E-Shop Application. All rights reserved.
  </footer>
</body>
</html>
```

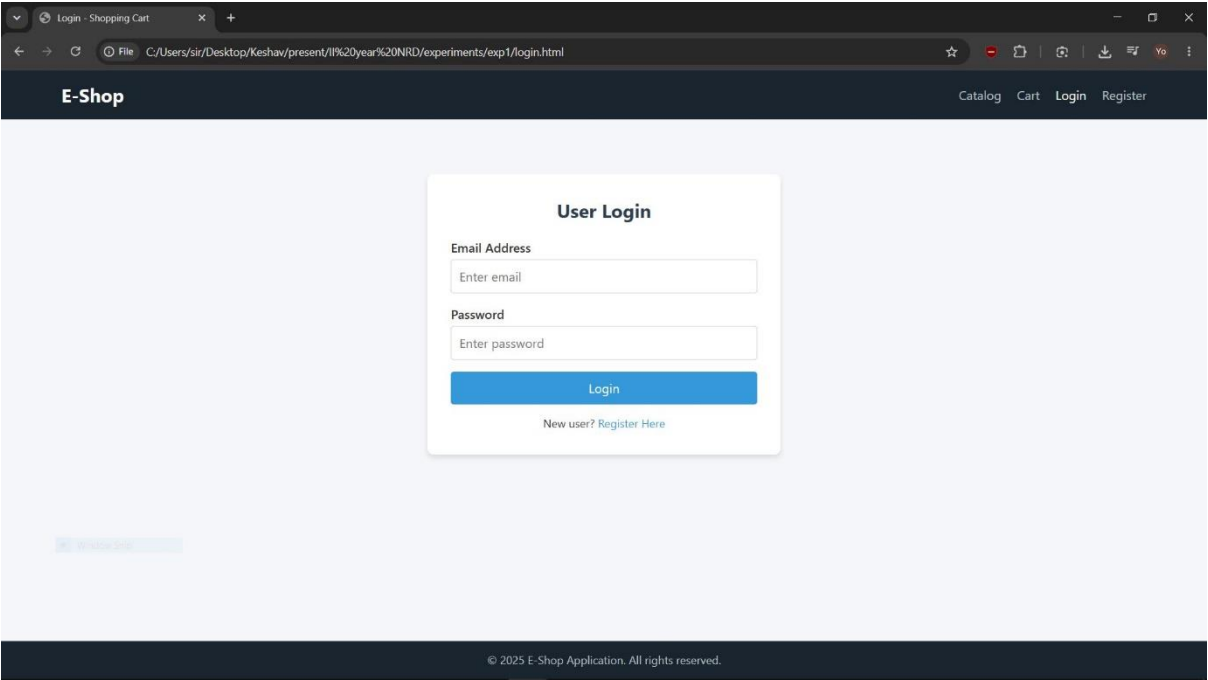


2. login.html (Login Page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login - Shopping Cart</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <nav class="navbar">
      <a href="catalog.html" class="brand">E-Shop</a>
      <ul class="nav-links">
        <li><a href="catalog.html">Catalog</a></li>
        <li><a href="cart.html">Cart</a></li>
        <li><a href="login.html" class="active">Login</a></li>
        <li><a href="register.html">Register</a></li>
      </ul>
    </nav>
  </header>

  <div class="container">
    <div class="form-container">
      <h2>User Login</h2>
      <form action="catalog.html" method="GET">
        <div class="form-group">
          <label for="email">Email Address</label>
          <input type="email" id="email" required placeholder="Enter email">
        </div>
        <div class="form-group">
          <label for="password">Password</label>
          <input type="password" id="password" required placeholder="Enter password">
        </div>
        <button type="submit" class="btn">Login</button>
      </form>
      <div class="form-footer">
        New user? <a href="register.html">Register Here</a>
      </div>
    </div>

    <footer>
      &copy; 2025 E-Shop Application. All rights reserved.
    </footer>
  </body>
</html>
```



3. catalog.html (Product Catalogue Page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Catalog - Shopping Cart</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <nav class="navbar">
      <a href="catalog.html" class="brand">E-Shop</a>
      <ul class="nav-links">
        <li><a href="catalog.html" class="active">Catalog</a></li>
        <li><a href="cart.html">Cart</a></li>
        <li><a href="login.html">Login</a></li>
        <li><a href="register.html">Register</a></li>
      </ul>
    </nav>
  </header>

  <div class="container">
    <h2 style="margin-bottom: 25px; color: #2c3e50;">Products Catalog</h2>

    <div class="catalog-grid">
      <!-- Product Item 1 -->
      <div class="product-card">
        <div class="product-image">Product Image 1</div>
        <div class="product-info">
          <h3>Wireless Headphones</h3>
          <p>Noise-canceling over-ear wireless headphones with long battery life.</p>
        </div>
        <div class="product-price">$99.99</div>
        <button class="btn" onclick="location.href='cart.html'">Add to Cart</button>
      </div>

      <!-- Product Item 2 -->
      <div class="product-card">
        <div class="product-image">Product Image 2</div>
        <div class="product-info">
          <h3>Smart Watch</h3>
          <p>Water-resistant fitness tracker smart watch with heart rate monitor.</p>
        </div>
        <div class="product-price">$149.99</div>
        <button class="btn" onclick="location.href='cart.html'">Add to Cart</button>
      </div>

      <!-- Product Item 3 -->
      <div class="product-card">
        <div class="product-image">Product Image 3</div>
        <div class="product-info">
          <h3>Mechanical Keyboard</h3>
          <p>RGB backlit mechanical keyboard with customizable tactile switches.</p>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

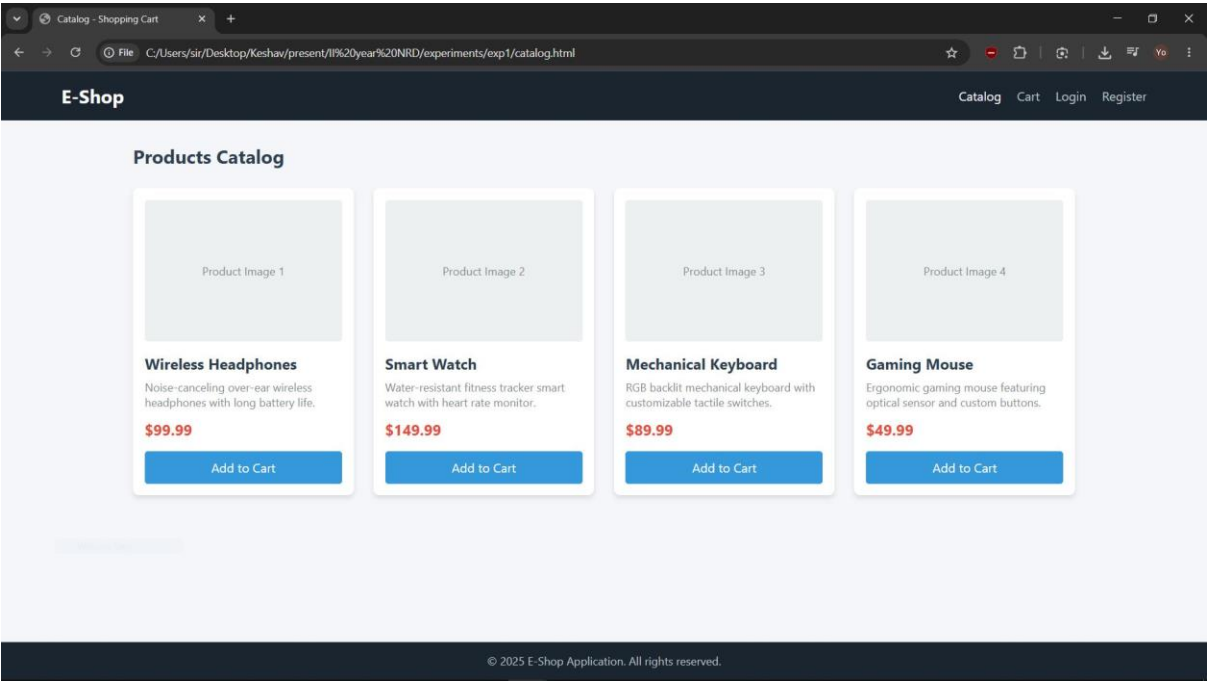
```

</div>
<div class="product-price">$89.99</div>
<button class="btn" onclick="location.href='cart.html'">Add to Cart</button>
</div>

<!-- Product Item 4 -->
<div class="product-card">
  <div class="product-image">Product Image 4</div>
  <div class="product-info">
    <h3>Gaming Mouse</h3>
    <p>Ergonomic gaming mouse featuring optical sensor and custom buttons.</p>
  </div>
  <div class="product-price">$49.99</div>
  <button class="btn" onclick="location.href='cart.html'">Add to Cart</button>
</div>
</div>
</div>

<footer>
  &copy; 2025 E-Shop Application. All rights reserved.
</footer>
</body>
</html>

```



4. cart.html (Shopping Cart Page)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Cart - Shopping Cart</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <nav class="navbar">
      <a href="catalog.html" class="brand">E-Shop</a>
      <ul class="nav-links">
        <li><a href="catalog.html">Catalog</a></li>
        <li><a href="cart.html" class="active">Cart</a></li>
        <li><a href="login.html">Login</a></li>
        <li><a href="register.html">Register</a></li>
      </ul>
    </nav>
  </header>

  <div class="container">
    <h2 style="margin-bottom: 25px; color: #2c3e50;">Shopping Cart</h2>

    <div class="cart-layout">
      <!-- Left Side: Cart Items -->
      <div class="cart-items">
        <!-- Cart Item 1 -->
        <div class="cart-item">
          <div class="cart-item-info">
            <div class="cart-item-img"></div>
            <div class="cart-item-details">
              <h4>Wireless Headphones</h4>
              <p>Unit Price: $99.99</p>
            </div>
          </div>
          <div class="cart-item-actions">
            <input type="number" class="quantity-input" value="1" min="1">
            <button class="remove-btn">Remove</button>
          </div>
        </div>

        <!-- Cart Item 2 -->
        <div class="cart-item">
          <div class="cart-item-info">
            <div class="cart-item-img"></div>
            <div class="cart-item-details">
              <h4>Mechanical Keyboard</h4>
              <p>Unit Price: $89.99</p>
            </div>
          </div>
          <div class="cart-item-actions">
            <input type="number" class="quantity-input" value="1" min="1">
```

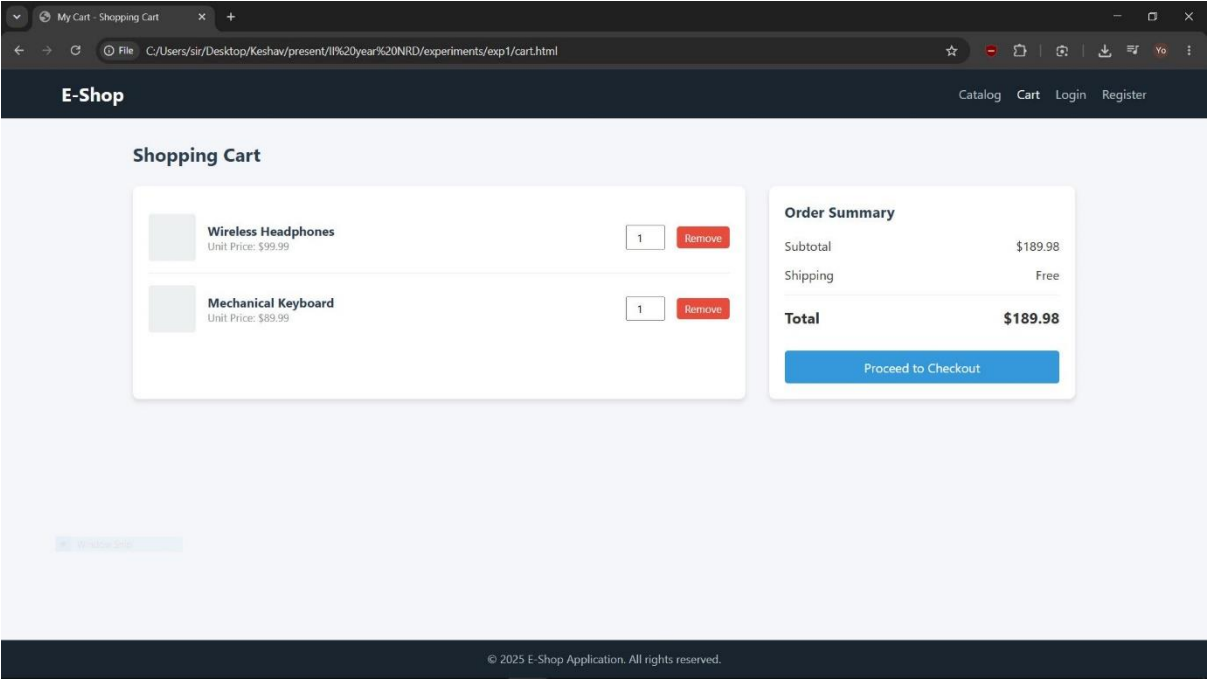
```

        <button class="remove-btn">Remove</button>
      </div>
    </div>
  </div>

  <!-- Right Side: Order Summary -->
  <div class="cart-summary">
    <h3 style="margin-bottom: 20px; color: #2c3e50;">Order Summary</h3>
    <div class="summary-row">
      <span>Subtotal</span>
      <span>$189.98</span>
    </div>
    <div class="summary-row">
      <span>Shipping</span>
      <span>Free</span>
    </div>
    <div class="summary-row summary-total">
      <span>Total</span>
      <span>$189.98</span>
    </div>
    <button class="btn" style="margin-top: 15px;">Proceed to Checkout</button>
  </div>
</div>

<footer>
  &copy; 2025 E-Shop Application. All rights reserved.
</footer>
</body>
</html>

```



Experiment 2

Objective: Make the shopping cart web application (containing registration, login, catalog, and cart pages) responsive using the Bootstrap framework.

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Experiment 2: Bootstrap Shopping App</title>

  <!-- 1. Include Bootstrap 5 CSS via CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

  <style>
    /* Minimal custom CSS to handle dynamic page displaying and footer pinning */
    body {
      display: flex;
      flex-direction: column;
      min-height: 100vh;
    }
    .page {
      display: none; /* Hide pages by default */
    }
    .page.active {
      display: block; /* Show the active page */
    }
    footer {
      margin-top: auto; /* Push footer to the bottom of the viewport */
    }
  </style>
</head>
<body>

  <!-- -----
  BOOTSTRAP NAVIGATION BAR
  - 'navbar-expand-lg' ensures it expands to a row on large screens and collapses into a burger menu on
  smaller devices.
  - 'navbar-dark bg-dark' applies Bootstrap's default dark mode styling.
  ----- -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand fw-bold" href="#" onclick="showPage('catalog', this)">E-Shop</a>

      <!-- Mobile Toggle Button (Burger Menu) -->
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav"
aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>

      <!-- Navbar Links -->
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto"> <!-- 'ms-auto' pushes items to the right side -->
          <li class="nav-item">
            <a class="nav-link active nav-btn" href="#" onclick="showPage('catalog', this)">Catalog</a>
```

```

    </li>
    <li class="nav-item">
      <a class="nav-link nav-btn" href="#" onclick="showPage('cart', this)">Cart</a>
    </li>
    <li class="nav-item">
      <a class="nav-link nav-btn" href="#" onclick="showPage('login', this)">Login</a>
    </li>
    <li class="nav-item">
      <a class="nav-link nav-btn" href="#" onclick="showPage('register', this)">Register</a>
    </li>
  </ul>
</div>
</div>
</nav>

```

<!-- Main Container -->

```
<main class="container my-5">
```

```
<!-- -----
```

SECTION: PRODUCT CATALOG (Bootstrap Grid)

- 'row-cols-1': 1 item per row on mobile.
- 'row-cols-md-3': 3 items per row on medium screens and larger.
- 'g-4': Applies a gap spacing of 4 units between grid items.

```
----- -->
```

```
<section id="catalog" class="page active">
```

```
<h3 class="mb-4">Products</h3>
```

```
<div class="row row-cols-1 row-cols-md-3 g-4">
```

<!-- Product Card 1 -->

```
<div class="col">
```

```
<div class="card h-100 shadow-sm"> <!-- 'h-100' ensures all cards in a row have equal height -->
```

```
<div class="bg-light text-center py-5 text-muted" style="font-size: 1.1rem;">
```

Wireless Headset

```
</div>
```

```
<div class="card-body d-flex flex-column justify-content-between">
```

```
<h5 class="card-title">Product A</h5>
```

```
<p class="card-text text-muted">High-quality wireless headphones with active noise
```

cancellation.</p>

```
<h6 class="text-danger fw-bold">$25.00</h6>
```

```
<button class="btn btn-success mt-2" onclick="navigateToCart()">Add to Cart</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

<!-- Product Card 2 -->

```
<div class="col">
```

```
<div class="card h-100 shadow-sm">
```

```
<div class="bg-light text-center py-5 text-muted" style="font-size: 1.1rem;">
```

Mechanical Keyboard

```
</div>
```

```
<div class="card-body d-flex flex-column justify-content-between">
```

```
<h5 class="card-title">Product B</h5>
```

```
<p class="card-text text-muted">RGB mechanical gaming keyboard with responsive clicky
```

switches.</p>

```
<h6 class="text-danger fw-bold">$45.00</h6>
```

```
<button class="btn btn-success mt-2" onclick="navigateToCart()">Add to Cart</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

```

<!-- Product Card 3 -->
<div class="col">
  <div class="card h-100 shadow-sm">
    <div class="bg-light text-center py-5 text-muted" style="font-size: 1.1rem;">
      Gaming Mouse
    </div>
    <div class="card-body d-flex flex-column justify-content-between">
      <h5 class="card-title">Product C</h5>
      <p class="card-text text-muted">Ergonomic lightweight wired gaming mouse with precision
sensor.</p>
      <h6 class="text-danger fw-bold">$15.00</h6>
      <button class="btn btn-success mt-2" onclick="navigateToCart()">Add to Cart</button>
    </div>
  </div>
</div>
</section>

```

```

<!-- -----
SECTION: CART PAGE (Bootstrap Responsive Split Column Layout)
- 'col-lg-8' takes up 8/12 grid spaces on large screens.
- 'col-lg-4' takes up remaining 4/12 spaces for summary.
----- -->

```

```

<section id="cart" class="page">
  <h3 class="mb-4">Shopping Cart</h3>
  <div class="row g-4">

    <!-- Left: Selected Items List -->
    <div class="col-lg-8">
      <ul class="list-group shadow-sm">
        <li class="list-group-item d-flex justify-content-between align-items-center py-3">
          <div>
            <h6 class="mb-0">Product A</h6>
            <small class="text-muted">Quantity: 1</small>
          </div>
          <span class="fw-bold text-dark">$25.00</span>
        </li>
        <li class="list-group-item d-flex justify-content-between align-items-center py-3">
          <div>
            <h6 class="mb-0">Product B</h6>
            <small class="text-muted">Quantity: 1</small>
          </div>
          <span class="fw-bold text-dark">$45.00</span>
        </li>
      </ul>
    </div>

```

```

    <!-- Right: Price Summary Panel -->
    <div class="col-lg-4">
      <div class="card p-3 shadow-sm bg-light">
        <h5 class="mb-3">Order Summary</h5>
        <div class="d-flex justify-content-between mb-2">
          <span>Subtotal</span>
          <span>$70.00</span>
        </div>
        <div class="d-flex justify-content-between mb-3 text-success">
          <span>Shipping</span>
          <span>Free</span>
        </div>

```

```

        <hr>
        <div class="d-flex justify-content-between mb-3 fw-bold fs-5">
            <span>Total</span>
            <span>$70.00</span>
        </div>
        <button class="btn btn-primary w-100">Proceed to Checkout</button>
    </div>
</div>
</section>

<!-- -----
SECTION: USER LOGIN (Bootstrap Card & Form styling)
- 'col-md-6 mx-auto' centers a 6-grid width box on tablet/desktop devices.
----- -->
<section id="login" class="page">
    <div class="row">
        <div class="col-md-6 mx-auto">
            <div class="card p-4 shadow">
                <h4 class="text-center mb-4">Login</h4>
                <form onsubmit="event.preventDefault(); showPage('catalog', document.querySelector('.nav-
btn:nth-child(1)'));">
                    <div class="mb-3">
                        <label class="form-label">Email address</label>
                        <input type="email" class="form-control" required placeholder="name@example.com">
                    </div>
                    <div class="mb-3">
                        <label class="form-label">Password</label>
                        <input type="password" class="form-control" required placeholder="••••••••">
                    </div>
                    <button type="submit" class="btn btn-primary w-100">Sign In</button>
                </form>
            </div>
        </div>
    </div>
</section>

<!-- -----
SECTION: REGISTRATION FORM
----- -->
<section id="register" class="page">
    <div class="row">
        <div class="col-md-6 mx-auto">
            <div class="card p-4 shadow">
                <h4 class="text-center mb-4">Register</h4>
                <form onsubmit="event.preventDefault(); showPage('login', document.querySelector('.nav-
btn:nth-child(3)'));">
                    <div class="mb-3">
                        <label class="form-label">Full Name</label>
                        <input type="text" class="form-control" required placeholder="John Doe">
                    </div>
                    <div class="mb-3">
                        <label class="form-label">Email address</label>
                        <input type="email" class="form-control" required placeholder="john@example.com">
                    </div>
                    <div class="mb-3">
                        <label class="form-label">Password</label>
                        <input type="password" class="form-control" required placeholder="Create Password">
                    </div>
                </form>
            </div>
        </div>
    </div>
</section>

```

```

        <button type="submit" class="btn btn-success w-100">Sign Up</button>
    </form>
</div>
</div>
</div>
</section>

</main>

<!-- Footer Alignment -->
<footer class="bg-dark text-white text-center py-3 mt-auto">
    <p class="mb-0">&copy; 2025 E-Shop. Built using Bootstrap 5.</p>
</footer>

<!-- 2. Include Bootstrap 5 JS Bundle (Enables Navbar collapses, dropdowns, etc.) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

<!-- Page Switcher Script logic -->
<script>
    function showPage(pageId, navElement) {
        // Hide all pages by removing active class
        document.querySelectorAll('.page').forEach(page => page.classList.remove('active'));
        // Display targeted page
        document.getElementById(pageId).classList.add('active');

        // Set clicked nav element as visually active
        document.querySelectorAll('.nav-link').forEach(link => link.classList.remove('active'));
        if (navElement) {
            navElement.classList.add('active');
        }
    }

    // Special route helper for adding items and jumping to cart page view
    function navigateToCart() {
        const cartLink = document.querySelectorAll('.nav-link')[1];
        showPage('cart', cartLink);
    }
</script>
</body>
</html>

```

Register

Full Name

John Doe

Email address

john@example.com

Password

Create Password

Sign Up

Login

Email address

name@example.com

Password

Sign In

Products

Wireless Headset

Product A

High-quality wireless headphones with active noise cancellation.

\$25.00

Add to Cart

Mechanical Keyboard

Product B

RGB mechanical gaming keyboard with responsive clicky switches.

\$45.00

Add to Cart

Gaming Mouse

Product C

Ergonomic lightweight wired gaming mouse with precision sensor.

\$15.00

Add to Cart

Shopping Cart

Product A

Quantity: 1

\$25.00

Product B

Quantity: 1

\$45.00

Order Summary

Subtotal

\$70.00

Shipping

Free

Total

\$70.00

Proceed to Checkout

Experiment 3

Objective: Use JavaScript to perform client-side validation on the login and registration pages built in the previous experiments.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Experiment 3: Client-Side Validation</title>

  <!-- Include Bootstrap 5 CSS via CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

  <style>
    body {
      display: flex;
      flex-direction: column;
      min-height: 100vh;
    }
    .page {
      display: none; /* Hide pages by default */
    }
    .page.active {
      display: block; /* Show the active page */
    }
    footer {
      margin-top: auto; /* Push footer to the bottom of the viewport */
    }
  </style>
</head>
<body>

  <!-- Bootstrap Navigation Bar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand fw-bold" href="#" onclick="showPage('catalog', this)">E-Shop</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle
navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item">
            <a class="nav-link nav-btn" href="#" onclick="showPage('catalog', this)">Catalog</a>
          </li>
          <li class="nav-item">
            <a class="nav-link nav-btn" href="#" onclick="showPage('cart', this)">Cart</a>
          </li>
          <li class="nav-item">
```

```

        <a class="nav-link active nav-btn" href="#" onclick="showPage('login',
this)">Login</a>
    </li>
    <li class="nav-item">
        <a class="nav-link nav-btn" href="#" onclick="showPage('register', this)">Register</a>
    </li>
</ul>
</div>
</div>
</nav>

```

<!-- Main Container -->

```
<main class="container my-5">
```

```

<!-- -----
SECTION: USER LOGIN
----- -->

```

```
<section id="login" class="page active">
```

```
<div class="row">
```

```
<div class="col-md-6 mx-auto">
```

```
<div class="card p-4 shadow">
```

```
<h4 class="text-center mb-4">Login</h4>
```

<!-- 'novalidate' disables browser default tooltip validation so we can handle custom feedback -->

```
<form id="loginForm" novalidate>
```

```
<div class="mb-3">
```

```
<label class="form-label" for="loginEmail">Email address</label>
```

```
<input type="email" class="form-control" id="loginEmail"
```

```
placeholder="name@example.com">
```

```
<div class="invalid-feedback">Please enter a valid email address (e.g.,
name@example.com).</div>
```

```
</div>
```

```
<div class="mb-3">
```

```
<label class="form-label" for="loginPassword">Password</label>
```

```
<input type="password" class="form-control" id="loginPassword"
```

```
placeholder="••••••••">
```

```
<div class="invalid-feedback">Password cannot be empty.</div>
```

```
</div>
```

```
<button type="submit" class="btn btn-primary w-100">Sign In</button>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</section>
```

```

<!-- -----
SECTION: REGISTRATION FORM
----- -->

```

```
<section id="register" class="page">
```

```
<div class="row">
```

```
<div class="col-md-6 mx-auto">
```

```
<div class="card p-4 shadow">
```

```
<h4 class="text-center mb-4">Register</h4>
```

```
<form id="registerForm" novalidate>
```

```

        <div class="mb-3">
            <label class="form-label" for="regName">Full Name</label>
            <input type="text" class="form-control" id="regName" placeholder="John Doe">
            <div class="invalid-feedback">Name should contain only letters and spaces (min
3 characters).</div>
        </div>
        <div class="mb-3">
            <label class="form-label" for="regEmail">Email address</label>
            <input type="email" class="form-control" id="regEmail"
placeholder="john@example.com">
            <div class="invalid-feedback">Please enter a valid email address.</div>
        </div>
        <div class="mb-3">
            <label class="form-label" for="regPassword">Password</label>
            <input type="password" class="form-control" id="regPassword"
placeholder="Create Password">
            <div class="invalid-feedback">Password must be at least 6 characters long and
contain at least one number.</div>
        </div>
        <button type="submit" class="btn btn-success w-100">Sign Up</button>
    </form>
</div>
</div>
</div>
</section>

```

```

<!-- -----
SECTION: PRODUCT CATALOG (Placeholder from Exp 2)
----- -->

```

```

<section id="catalog" class="page">
    <h3 class="mb-4">Products</h3>
    <div class="alert alert-info">Login successful! Catalog is now unlocked.</div>
</section>

```

```

<!-- -----
SECTION: CART PAGE (Placeholder from Exp 2)
----- -->

```

```

<section id="cart" class="page">
    <h3 class="mb-4">Shopping Cart</h3>
</section>

```

```

</main>

```

```

<!-- Footer -->

```

```

<footer class="bg-dark text-white text-center py-3 mt-auto">
    <p class="mb-0">&copy; 2025 E-Shop. Built using Bootstrap 5 with JS Validation.</p>
</footer>

```

```

<!-- Bootstrap 5 JS Bundle -->

```

```

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

```

```

<!-- -----
VALIDATION AND PAGE NAVIGATION LOGIC

```

```
----->
<script>
```

```
// Form Selector References
```

```
const loginForm = document.getElementById('loginForm');
```

```
const registerForm = document.getElementById('registerForm');
```

```
/* -----
   Client-Side Validation Helper Functions
   ----- */
```

```
// Checks if email matches standard pattern (e.g., name@domain.com)
```

```
function validateEmail(email) {
```

```
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
    return emailRegex.test(email);
```

```
}
```

```
// Checks if Name only contains letters and spaces (min 3 chars)
```

```
function validateName(name) {
```

```
    const nameRegex = /^[a-zA-Z\s]{3,}$/;
```

```
    return nameRegex.test(name);
```

```
}
```

```
// Checks if password contains at least 6 characters and 1 digit
```

```
function validatePasswordStrength(password) {
```

```
    const passwordRegex = /^(?=.*[0-9]).{6,}$/;
```

```
    return passwordRegex.test(password);
```

```
}
```

```
// Applies Bootstrap visual feedback classes (is-valid / is-invalid)
```

```
function markInputStatus(inputElement, isValid) {
```

```
    if (isValid) {
```

```
        inputElement.classList.remove('is-invalid');
```

```
        inputElement.classList.add('is-valid');
```

```
    } else {
```

```
        inputElement.classList.remove('is-valid');
```

```
        inputElement.classList.add('is-invalid');
```

```
    }
```

```
}
```

```
/* -----
   Form Submission Event Listeners with Validation Controls
   ----- */
```

```
// Event listener for Login Form Validation
```

```
loginForm.addEventListener('submit', function (event) {
```

```
    event.preventDefault(); // Stop standard form submission reload
```

```
    const emailInput = document.getElementById('loginEmail');
```

```
    const passwordInput = document.getElementById('loginPassword');
```

```
// Evaluate email
```

```
const isEmailValid = validateEmail(emailInput.value.trim());
```

```
markInputStatus(emailInput, isEmailValid);
```

```
// Evaluate simple empty field state for password
const isPasswordValid = passwordInput.value.trim() !== '';
markInputStatus(passwordInput, isPasswordValid);
```

```
// If both validations pass, redirect user to the Catalog Page
if (isEmailValid && isPasswordValid) {
    alert("Login successful!");
    showPage('catalog', document.querySelectorAll('.nav-link')[0]);
    loginForm.reset();
    emailInput.classList.remove('is-valid');
    passwordInput.classList.remove('is-valid');
}
});
```

```
// Event listener for Registration Form Validation
registerForm.addEventListener('submit', function (event) {
    event.preventDefault(); // Stop page reload
```

```
const nameInput = document.getElementById('regName');
const emailInput = document.getElementById('regEmail');
const passwordInput = document.getElementById('regPassword');
```

```
// Run validation rules
const isNameValid = validateName(nameInput.value.trim());
const isEmailValid = validateEmail(emailInput.value.trim());
const isPasswordValid = validatePasswordStrength(passwordInput.value.trim());
```

```
// Apply visual styles to inputs based on results
markInputStatus(nameInput, isNameValid);
markInputStatus(emailInput, isEmailValid);
markInputStatus(passwordInput, isPasswordValid);
```

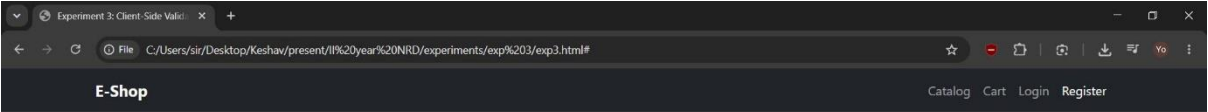
```
// If all fields are validated, redirect user to the Login page
if (isNameValid && isEmailValid && isPasswordValid) {
    alert("Registration successful! Please login.");
    showPage('login', document.querySelectorAll('.nav-link')[2]);
    registerForm.reset();
    nameInput.classList.remove('is-valid');
    emailInput.classList.remove('is-valid');
    passwordInput.classList.remove('is-valid');
}
});
```

```
/* -----
Dynamic Display Navigation logic
----- */
```

```
function showPage(pageId, navElement) {
    document.querySelectorAll('.page').forEach(page => page.classList.remove('active'));
    document.getElementById(pageId).classList.add('active');
```

```
document.querySelectorAll('.nav-link').forEach(link => link.classList.remove('active'));
if (navElement) {
    navElement.classList.add('active');
}
```

```
}
</script>
</body>
</html>
```



Register

Full Name

John Doe

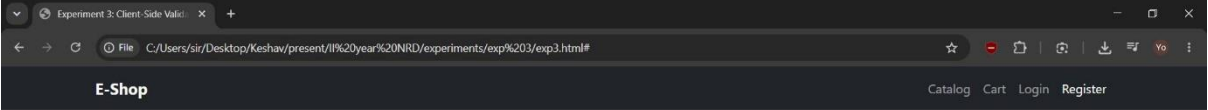
Email address

john@example.com

Password

Create Password

Sign Up



Register

Full Name

John Doe

Name should contain only letters and spaces (min 3 characters).

Email address

john@example.com

Please enter a valid email address.

Password

Create Password

Password must be at least 6 characters long and contain at least one number.

Sign Up

Login

Email address

name@example.com

Password

Sign In

Login

Email address

name@example.com

Please enter a valid email address (e.g., name@example.com).

Password

Password cannot be empty.

Sign In

Experiment 4

Objective: Explore the features of ES6 (arrow functions, callbacks, promises, async/await). Implement an application for reading weather information from openweathermap.org and display the information in the form of a graph on the web page.

Note: OpenWeatherMap requires an API Key

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Experiment 4: Live Weather Graph App</title>

  <!-- Bootstrap 5 CDN for styling -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- Chart.js CDN for displaying graphical representation -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body class="bg-light">

  <div class="container my-5">
    <div class="row">
      <div class="col-md-8 mx-auto">
        <div class="card p-4 shadow-sm bg-white">
          <h2 class="text-center mb-4 text-primary">Live Weather Graph Tracker</h2>

          <!-- ES6 Concept Demonstration Panel -->
          <div class="alert alert-info py-2">
            <small><strong>ES6 Features Used:</strong> Arrow Functions, Template Literals,
Promises, and <code>async/await</code> block structures.</small>
          </div>

          <!-- Input search control -->
          <div class="input-group mb-3">
            <span class="input-group-text bg-light text-dark">City Name</span>
            <input type="text" id="cityInput" class="form-control" placeholder="Enter City Name
(e.g., Hyderabad, London)" value="Hyderabad">
            <button class="btn btn-primary" id="getWeatherBtn">Fetch Live Weather</button>
          </div>

          <!-- Error notification display panel -->
          <div id="errorMessage" class="text-danger mb-3 fw-bold text-center"></div>

          <!-- Card details containing the text representation -->
          <div id="weatherDetails" class="mb-4 text-center d-none">
            <h4 id="cityNameDisplay" class="fw-bold mb-1">City Name</h4>
            <p id="tempDisplay" class="fs-3 text-danger mb-0">-- &deg;C</p>
            <p id="descDisplay" class="text-muted text-capitalize">--</p>
          </div>
        </div>
      </div>
    </div>
  </div>
```

```

        <!-- Canvas element where Chart.js will draw the data graph -->
        <div class="chart-container" style="position: relative; height:300px; width:100%">
            <canvas id="weatherChart"></canvas>
        </div>
    </div>
</div>
</div>
</div>

```

```

<!-------
ES6 CODE LOGIC IMPLEMENTATION WITH EMBEDDED API KEY
----->

```

```

<script>

```

```

    // Pre-configured API Key provided by the user

```

```

    const API_KEY = "53d57106482f8bfecbba5a1d4d4754896";

```

```

    let weatherChartInstance = null;

```

```

    // ES6 Arrow Function: Get elements cleanly from the DOM

```

```

    const getEl = id => document.getElementById(id);

```

```

    /* -----

```

1. ES6 CALLBACKS

- The renderChart function is passed as a callback handler to process the UI rendering sequence once weather data has been retrieved.

```

    ----- */

```

```

    const updateWeatherUI = (data, callback) => {

```

```

        getEl("weatherDetails").classList.remove("d-none");

```

```

        getEl("cityNameDisplay").textContent = `${data.name}, ${data.sys.country}`;

```

```

        getEl("tempDisplay").innerHTML = `${Math.round(data.main.temp)} &deg;C`;

```

```

        getEl("descDisplay").textContent = data.weather[0].description;

```

```

    // Execute the callback function to render the graphical information

```

```

        callback(data);

```

```

    };

```

```

    /* -----

```

2. ES6 PROMISES & ASYNC/AWAIT WITH LIVE FETCH

- 'async' keyword identifies asynchronous processing block.

- 'await' pauses execution until the Fetch API Promise is resolved.

```

    ----- */

```

```

    const fetchWeatherData = async () => {

```

```

        const city = getEl("cityInput").value.trim();

```

```

        getEl("errorMessage").textContent = ""; // Clear existing error logs

```

```

        if (!city) {

```

```

            getEl("errorMessage").textContent = "Please enter a city name.";

```

```

            return;

```

```

        }

```

```

    // API Endpoint structured with Live metrics configurations

```

```

    const url =

```

```

`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${API_KEY}&units=metric`;

```

```
try {  
  // Fetch returns a Promise. We use await to unpack the Response object  
  const response = await fetch(url);
```

```
  if (!response.ok) {  
    throw new Error("City not found or API limits exceeded.");  
  }
```

```
  const data = await response.json();
```

```
  // Process and pass data along with callback function for graph plotting  
  updateWeatherUI(data, renderGraphCallback);
```

```
} catch (error) {  
  // Handle Promise rejection/error state  
  getEl("errorMessage").textContent = `Error: ${error.message}`;  
}  
};
```

```
/* -----  
3. CHART RENDERING VIA CALLBACK  
----- */
```

```
const renderGraphCallback = (data) => {  
  const ctx = getEl('weatherChart').getContext('2d');
```

```
  // Dynamically simulate a temperature timeline graph centered around the current temp  
  const baseTemp = data.main.temp;  
  const labels = ['06:00 AM', '09:00 AM', '12:00 PM', '03:00 PM', '06:00 PM', '09:00 PM'];  
  const temperatures = [  
    baseTemp - 3, // cooler morning temp  
    baseTemp - 1,  
    baseTemp,    // current temp  
    baseTemp + 2, // peak afternoon temp  
    baseTemp - 2,  
    baseTemp - 4 // night temp  
  ];
```

```
  // If a previous chart instance exists, destroy it before rendering a new one  
  if (weatherChartInstance) {  
    weatherChartInstance.destroy();  
  }
```

```
  // Create new Chart instance using Chart.js  
  weatherChartInstance = new Chart(ctx, {  
    type: 'line',  
    data: {  
      labels: labels,  
      datasets: [{  
        label: `Hourly Temp Track for ${data.name} (°C)`,  
        data: temperatures,  
        borderColor: 'rgb(54, 162, 235)',  
        backgroundColor: 'rgba(54, 162, 235, 0.2)',  
        borderWidth: 2,  
        fill: true,
```

```

        tension: 0.3 // Smooth curves
    }}
},
options: {
    responsive: true,
    maintainAspectRatio: false,
    scales: {
        y: {
            beginAtZero: false,
            title: {
                display: true,
                text: 'Temperature (°C)'
            }
        }
    }
}
});
};

```

```

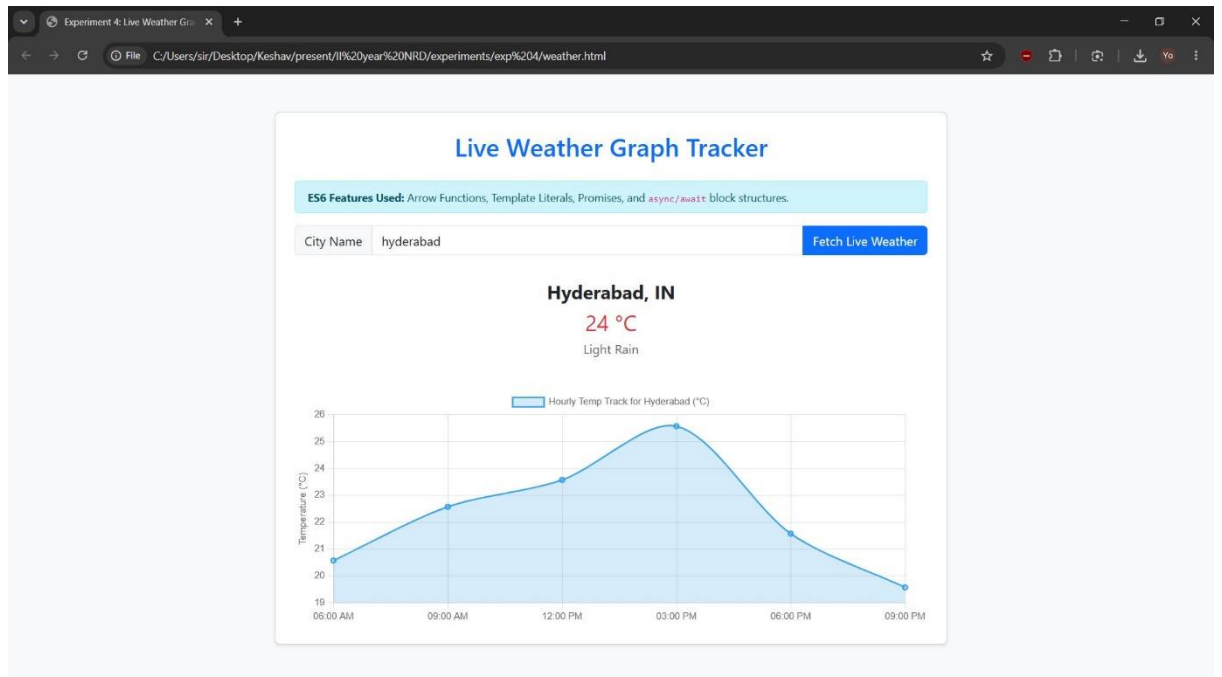
// Attach Click Event Handler to Button using ES6 Arrow Function syntax
getEl("getWeatherBtn").addEventListener("click", () => {
    fetchWeatherData();
});

```

```

// Trigger initial load automatically on startup
fetchWeatherData();
</script>
</body>
</html>

```



Experiment 5

Objective: Develop a Java standalone application that connects with a database (MySQL) and performs CRUD (Create, Read, Update, Delete) operations on database tables.

Step 1: Create the Database Table in MySQL

Run the following SQL commands in your MySQL Command Line Client or database management tool (like phpMyAdmin or MySQL Workbench) to set up the test database and table:

```
-- Create a database named 'college'
CREATE DATABASE college;

-- Switch to the 'college' database
USE college;

-- Create a table named 'student'
CREATE TABLE student (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    age INT NOT NULL
);
```

Step 2: Java Implementation Code (StudentCrudApp.java)

This console-driven application connects to the MySQL database and presents an interactive menu to perform **Create, Read, Update, and Delete** operations.

Note: Ensure you have added the MySQL Connector/J driver .jar file to your Java project build path or classpath.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

public class StudentCrudApp {

    // Database Connection Parameters
    private static final String DB_URL = "jdbc:mysql://localhost:3306/college";
    private static final String DB_USER = "root";    // Change to your MySQL username
    private static final String DB_PASSWORD = "password"; // Change to your MySQL password

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Connection conn = null;

        try {
            // 1. Register the JDBC driver and establish database connection
            System.out.println("Connecting to database...");
```

```
conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
System.out.println("Connection successful!");
```

```
while (true) {
    // Display Interaction Menu to User
    System.out.println("\n--- STUDENT CRUD OPERATIONS ---");
    System.out.println("1. Create (Insert Student)");
    System.out.println("2. Read (View All Students)");
    System.out.println("3. Update (Modify Student Email)");
    System.out.println("4. Delete (Remove Student)");
    System.out.println("5. Exit");
    System.out.print("Enter your choice: ");

    int choice = scanner.nextInt();
    scanner.nextLine(); // Clear the buffer input newline

    switch (choice) {
        case 1:
            createStudent(conn, scanner);
            break;
        case 2:
            readStudents(conn);
            break;
        case 3:
            updateStudent(conn, scanner);
            break;
        case 4:
            deleteStudent(conn, scanner);
            break;
        case 5:
            System.out.println("Exiting Application. Database connection closed.");
            return;
        default:
            System.out.println("Invalid selection. Try again.");
    }
}
```

```
} catch (SQLException e) {
    System.err.println("Database Error: " + e.getMessage());
} finally {
    // Clean up and close connection scanner resources safely
    try {
        if (conn != null) conn.close();
        scanner.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

```
/* -----
1. CREATE (Insert Data)
Using PreparedStatement prevents SQL injection attacks
----- */
```

```

private static void createStudent(Connection conn, Scanner scanner) throws SQLException {
    String sql = "INSERT INTO student (name, email, age) VALUES (?, ?, ?)";

    System.out.print("Enter Student Name: ");
    String name = scanner.nextLine();
    System.out.print("Enter Student Email: ");
    String email = scanner.nextLine();
    System.out.print("Enter Student Age: ");
    int age = scanner.nextInt();

    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, name);
        stmt.setString(2, email);
        stmt.setInt(3, age);

        int rowsInserted = stmt.executeUpdate();
        if (rowsInserted > 0) {
            System.out.println("Record created successfully!");
        }
    }
}

```

```

/* -----
2. READ (Select / View Data)
Uses Statement and ResultSet to iterate over rows in database
----- */

```

```

private static void readStudents(Connection conn) throws SQLException {
    String sql = "SELECT * FROM student";

    try (Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        System.out.println("\n-----");
        System.out.printf("%-5s | %-20s | %-20s | %-5s\n", "ID", "Name", "Email", "Age");
        System.out.println("-----");

        boolean recordsFound = false;
        while (rs.next()) {
            recordsFound = true;
            int id = rs.getInt("id");
            String name = rs.getString("name");
            String email = rs.getString("email");
            int age = rs.getInt("age");

            System.out.printf("%-5d | %-20s | %-20s | %-5d\n", id, name, email, age);
        }

        if (!recordsFound) {
            System.out.println("No records found in the table.");
        }
        System.out.println("-----");
    }
}

```

```
/* -----
```

3. UPDATE (Modify Data)

Using SQL UPDATE statements targeting a specific record ID

```
----- */
```

```
private static void updateStudent(Connection conn, Scanner scanner) throws SQLException {  
    String sql = "UPDATE student SET email = ? WHERE id = ?";
```

```
    System.out.print("Enter Student ID to update: ");
```

```
    int id = scanner.nextInt();
```

```
    scanner.nextLine(); // Clear the input buffer
```

```
    System.out.print("Enter New Email: ");
```

```
    String newEmail = scanner.nextLine();
```

```
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
```

```
        stmt.setString(1, newEmail);
```

```
        stmt.setInt(2, id);
```

```
        int rowsUpdated = stmt.executeUpdate();
```

```
        if (rowsUpdated > 0) {
```

```
            System.out.println("Record updated successfully!");
```

```
        } else {
```

```
            System.out.println("Record update failed. Student ID not found.");
```

```
        }
```

```
    }
```

```
}
```

```
/* -----
```

4. DELETE (Remove Data)

Using SQL DELETE statement to drop student records

```
----- */
```

```
private static void deleteStudent(Connection conn, Scanner scanner) throws SQLException {  
    String sql = "DELETE FROM student WHERE id = ?";
```

```
    System.out.print("Enter Student ID to delete: ");
```

```
    int id = scanner.nextInt();
```

```
    try (PreparedStatement stmt = conn.prepareStatement(sql)) {
```

```
        stmt.setInt(1, id);
```

```
        int rowsDeleted = stmt.executeUpdate();
```

```
        if (rowsDeleted > 0) {
```

```
            System.out.println("Record deleted successfully!");
```

```
        } else {
```

```
            System.out.println("Record deletion failed. Student ID not found.");
```

```
        }
```

```
    }
```

```
}
```

```
}
```

OUTPUT:

Connecting to database...

Connection successful!

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)
- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 2

ID	Name	Email	Age
----	------	-------	-----

No records found in the table.

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)
- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 1

Enter Student Name: ram

Enter Student Email: ram@gmail.com

Enter Student Age: 40

Record created successfully!

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)

- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 2

ID	Name	Email	Age
1	ram	ram@gmail.com	40

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)
- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 3

Enter Student ID to update: 1

Enter New Email: rama@gmail.com

Record updated successfully!

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)
- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 2

ID	Name	Email	Age
1	ram	rama@gmail.com	40

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)
- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 4

Enter Student ID to delete: 1

Record deleted successfully!

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)
- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 2

ID	Name	Email	Age
----	------	-------	-----

No records found in the table.

--- STUDENT CRUD OPERATIONS ---

- 1. Create (Insert Student)
- 2. Read (View All Students)
- 3. Update (Modify Student Email)
- 4. Delete (Remove Student)
- 5. Exit

Enter your choice: 5

Exiting Application. Database connection closed.

Experiment 6

Objective: Create an XML file for a bookstore and validate it using both Document Type Definition (DTD) and XML Schema Definition (XSD).

File 1: The XML Data File (bookstore.xml)

This file contains the database records of books in the bookstore. It includes references pointing to both the DTD file (for structure validation) and the XSD file (for strict data-type validation).

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  1. DOCTYPE declaration: Points to the external 'bookstore.dtd' file
    to validate the structural elements of this XML.
-->
<!DOCTYPE bookstore SYSTEM "bookstore.dtd">

<!--
  2. Bookstore root element:
    'xsi:noNamespaceSchemaLocation' links this XML instance directly to
    the schema definition file 'bookstore.xsd' for strict data-type
    checking.
-->
<bookstore xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xsi:noNamespaceSchemaLocation="bookstore.xsd">

  <!-- Book Entry 1 -->
  <book id="B01" category="Programming">
    <title>Pro MERN Stack</title>
    <author>Vasan Subramanian</author>
    <year>2019</year>
    <price>49.99</price>
  </book>

  <!-- Book Entry 2 -->
  <book id="B02" category="Web Development">
    <title>Beginning HTML, CSS and JavaScript</title>
    <author>Jon Duckett</author>
    <year>2010</year>
    <price>29.99</price>
  </book>

  <!-- Book Entry 3 -->
  <book id="B03" category="Fiction">
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <year>1925</year>
    <price>14.95</price>
  </book>

</bookstore>
```

File 2: Structural Validation (bookstore.dtd)

This DTD file sets up basic structure rules. It dictates which sub-elements must appear inside each tag, their order, and whether elements are optional, required, or repetitive.

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  DTD Rules:
  - 'bookstore' contains one or more 'book' elements.
  - Each 'book' contains exactly one title, author, year, and price.
-->
<!ELEMENT bookstore (book+)>
<!ELEMENT book (title, author, year, price)>

<!-- Sub-elements definition -->
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT price (#PCDATA)>

<!--
  1. Root Element Bookstore Attributes:
  We declare XML schema namespace attributes as CDATA #IMPLIED (optional)
  so that strict DTD parser checks do not fail when XSD attributes are
  present.
-->
<!ATTLIST bookstore
  xmlns:xsi CDATA #IMPLIED
  xsi:noNamespaceSchemaLocation CDATA #IMPLIED>

<!--
  2. Book Element Attributes
-->
<!ATTLIST book
  id ID #REQUIRED
  category CDATA #REQUIRED>
```

File 3: Type Validation Schema (bookstore.xsd)

While DTD only validates basic structure, this XSD (XML Schema Definition) enforces strict structural and data-type validations (e.g., verifying that the year is a whole integer and the price is a decimal number).

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified">

  <!-- Define Bookstore as the Root Element -->
  <xs:element name="bookstore">
    <xs:complexType>
      <xs:sequence>
        <!-- Bookstore holds 1 to unbounded (unlimited) occurrences of 'book' -->
        <xs:element name="book" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <!-- 'title' is a standard string data type -->
              <xs:element name="title" type="xs:string"/>

              <!-- 'author' is a standard string data type -->
              <xs:element name="author" type="xs:string"/>

              <!-- 'year' is configured as a whole positive integer -->
              <xs:element name="year" type="xs:positiveInteger"/>

              <!-- 'price' is configured as a fractional decimal value -->
              <xs:element name="price" type="xs:decimal"/>
            </xs:sequence>

            <!-- Define Attribute elements for 'book' -->
            <xs:attribute name="id" type="xs:string" use="required"/>
            <xs:attribute name="category" type="xs:string" use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

XMLValidatorDual.java

```
import java.io.File;
import javax.xml.XMLConstants;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.stream.StreamSource;
import javax.xml.validation.Schema;
import javax.xml.validation.SchemaFactory;
import javax.xml.validation.Validator;
import org.xml.sax.ErrorHandler;
import org.xml.sax.SAXException;
import org.xml.sax.SAXParseException;

public class XMLValidatorDual {

    public static void main(String[] args) {
        String xmlFile = "bookstore.xml";
        String xsdFile = "bookstore.xsd";

System.out.println("=====");
        System.out.println("                DUAL XML VALIDATOR (DTD & XSD)
");

System.out.println("=====");

        // Step 1: Perform DTD Validation
        boolean isDtdValid = validateDTD(xmlFile);

        System.out.println(); // Formatter space

        // Step 2: Perform XSD Validation
        boolean isXsdValid = validateXSD(xmlFile, xsdFile);

System.out.println("=====");
        System.out.println("SUMMARY STATUS:");
        System.out.println("-> DTD Validation Status: " + (isDtdValid ?
"PASSED" : "FAILED"));
        System.out.println("-> XSD Validation Status: " + (isXsdValid ?
"PASSED" : "FAILED"));

System.out.println("=====");
    }

    /* -----
    1. DTD VALIDATION METHOD
    Uses DocumentBuilderFactory validation configuration
    ----- */
    private static boolean validateDTD(String xmlPath) {
        System.out.println("[Phase 1] Initiating DTD Validation...");
        try {
            DocumentBuilderFactory factory =
DocumentBuilderFactory.newInstance();

            // Instruct the parser to run DTD validation
            factory.setValidating(true);
            factory.setNamespaceAware(true);

            DocumentBuilder builder = factory.newDocumentBuilder();

            // Set custom error handler to catch DTD validation mismatches
```

```

        builder.setErrorHandler(new ErrorHandler() {
            @Override
            public void warning(SAXParseException e) {
                System.out.println("    Warning: " + e.getMessage());
            }

            @Override
            public void error(SAXParseException e) throws SAXException
        {
            System.err.println("    DTD Rule Violation: " +
e.getMessage() + " (Line: " + e.getLineNumber() + ")");
            throw e; // Terminate validation and mark as failed
        }

            @Override
            public void fatalError(SAXParseException e) throws
SAXException {
                System.err.println("    Fatal Error: " +
e.getMessage());
                throw e;
            }
        });

        // Parse XML (it will read and validate bookstore.dtd
internally)
        builder.parse(new File(xmlPath));
        System.out.println("[SUCCESS] XML matches structural DTD
successfully.");
        return true;

    } catch (SAXException e) {
        System.err.println("[FAILURE] DTD Validation failed due to a
structural mismatch.");
        return false;
    } catch (Exception e) {
        System.err.println("[ERROR] System Exception: " +
e.getMessage());
        return false;
    }
}

/* -----
2. XSD VALIDATION METHOD
Uses SchemaFactory and Validator configurations
----- */
private static boolean validateXSD(String xmlPath, String xsdPath) {
    System.out.println("[Phase 2] Initiating XSD Validation...");
    try {
        // Setup SchemaFactory for standard W3C schema validations
        SchemaFactory factory =
SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);

        // Load and parse XSD rules
        Schema schema = factory.newSchema(new File(xsdPath));

        // Build validator instance
        Validator validator = schema.newValidator();

        // Validate the XML target file
        validator.validate(new StreamSource(new File(xmlPath)));
        System.out.println("[SUCCESS] XML matches type-definition XSD
successfully.");
        return true;
    }
}

```

```
        } catch (SAXException e) {
            System.err.println("[FAILURE] XSD Validation failed: " +
e.getMessage());
            return false;
        } catch (Exception e) {
            System.err.println("[ERROR] System Exception: " +
e.getMessage());
            return false;
        }
    }
}
```

output:

```
=====

DUAL XML VALIDATOR (DTD & XSD)

=====

[Phase 1] Initiating DTD Validation...

[SUCCESS] XML matches structural DTD successfully.


[Phase 2] Initiating XSD Validation...

[SUCCESS] XML matches type-definition XSD successfully.

=====

SUMMARY STATUS:

-> DTD Validation Status: PASSED

-> XSD Validation Status: PASSED

=====
```

Output valuidation 1:
change

```
<!ELEMENT year (#PCDATA)>
```

to

```
<!-- <!ELEMENT year (#PCDATA)-->
```

```
=====

DUAL XML VALIDATOR (DTD & XSD)

=====

[Phase 1] Initiating DTD Validation...

DTD Rule Violation: Element type "year" must be declared. (Line: 20)

[FAILURE] DTD Validation failed due to a structural mismatch.


[Phase 2] Initiating XSD Validation...

[SUCCESS] XML matches type-definition XSD successfully.
```

=====

SUMMARY STATUS:

-> DTD Validation Status: FAILED

-> XSD Validation Status: PASSED

output validation 2 :

change XSD file

```
<xs:element name="title" type="xs:string"/>
to
```

```
<xs:element name="title" type="xs:int"/>
```

=====

DUAL XML VALIDATOR (DTD & XSD)

=====

[Phase 1] Initiating DTD Validation...

[SUCCESS] XML matches structural DTD successfully.

[Phase 2] Initiating XSD Validation...

[FAILURE] XSD Validation failed: cvc-datatype-valid.1.2.1: 'Pro MERN Stack' is not a valid value for 'integer'.

=====

SUMMARY STATUS:

-> DTD Validation Status: PASSED

-> XSD Validation Status: FAILED

=====

Experiment 7

Objective: Design a controller with a servlet that provides the interaction with the application developed in Experiment 1/2 and the database created in Experiment 5.

Architecture: MVC Concept

In this experiment, we implement the **Model-View-Controller (MVC)** architectural pattern:

- **View:** register.html (The Bootstrap front-end form where users enter data).
- **Controller:** RegisterServlet.java (Processes the HTTP POST request, extracts data, and coordinates with the database).
- **Model/Database:** MySQL college.student table (Stores the data permanently).

1. The View: register.html

This is the registration front-end form built in Experiment 2. It collects student details and posts them to the controller url /RegisterServlet.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration Portal</title>
  <!-- Include Bootstrap 5 CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">

  <div class="container my-5">
    <div class="row">
      <div class="col-md-6 mx-auto">
        <div class="card p-4 shadow-sm bg-white">
          <h3 class="text-center mb-4 text-primary">Student Registration</h3>

          <!-- Form actions point directly to the Servlet Controller URL -->
          <form action="RegisterServlet" method="POST">
            <div class="mb-3">
              <label class="form-label">Full Name</label>
              <input type="text" name="studentName" class="form-control"
                placeholder="Enter Full Name" required>
            </div>
            <div class="mb-3">
              <label class="form-label">Email Address</label>
              <input type="email" name="studentEmail" class="form-control"
                placeholder="Enter Email Address" required>
            </div>
            <div class="mb-3">
              <label class="form-label">Age</label>
```

```

        <input type="number" name="studentAge" class="form-control"
            placeholder="Enter Age" required min="10" max="100">
    </div>
    <button type="submit" class="btn btn-primary w-100">Register Student</button>
</form>

</div>
</div>
</div>
</div>

</body>
</html>

```

2. The Controller: RegisterServlet.java

This Servlet acts as the controller. It runs on a web server (like Apache Tomcat), reads the post request values, inserts them into the database using JDBC, and outputs an HTML response page.

```

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * Servlet controller mapping using@WebServlet Annotation (Servlet 3.0+)
 * This intercepts any HTTP POST requests directed to "/RegisterServlet"
 */
@WebServlet("/RegisterServlet")
public class RegisterServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // JDBC connection parameters targeting MySQL database configuration (from Exp 5)
    private static final String DB_URL = "jdbc:mysql://localhost:3306/college";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "password";

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Step 1: Set response content type to HTML
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();

        // Step 2: Read form input parameters sent from register.html
        String name = request.getParameter("studentName");
        String email = request.getParameter("studentEmail");
    }
}

```

```

String ageStr = request.getParameter("studentAge");
int age = Integer.parseInt(ageStr);

Connection conn = null;
PreparedStatement pstmt = null;
boolean isSuccess = false;

try {
    // Step 3: Register JDBC driver and connect to MySQL database instance
    Class.forName("com.mysql.cj.jdbc.Driver");
    conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

    // Step 4: Write Insert query with prepared statement
    String query = "INSERT INTO student (name, email, age) VALUES (?, ?, ?)";
    pstmt = conn.prepareStatement(query);
    pstmt.setString(1, name);
    pstmt.setString(2, email);
    pstmt.setInt(3, age);

    int result = pstmt.executeUpdate();
    if (result > 0) {
        isSuccess = true;
    }

} catch (ClassNotFoundException | SQLException e) {
    e.printStackTrace();
    // Output generic database error to client
    out.println("<h3 style='color:red;'>System Error occurred: " + e.getMessage() + "</h3>");
} finally {
    // Clean up statement and connection objects
    try {
        if (pstmt != null) pstmt.close();
        if (conn != null) conn.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// Step 5: Render response page feedback based on transaction status
out.println("<html>");
out.println("<head><title>Registration Status</title>");
out.println("<link");
href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css' rel='stylesheet'>");
out.println("</head>");
out.println("<body class='bg-light d-flex align-items-center justify-content-center'");
style='height:100vh;'>");
out.println("<div class='card p-4 text-center shadow-sm' style='max-width:400px;'>");

if (isSuccess) {
    out.println("<h3 class='text-success mb-3'>Success!</h3>");
    out.println("<p class='text-muted'>Student record was stored in the database.</p>");
    out.println("<p><strong>Name:</strong> " + name + "<br><strong>Email:</strong> " + email
+ "</p>");
} else {

```

```

        out.println("<h3 class='text-danger mb-3'>Registration Failed</h3>");
        out.println("<p class='text-muted'>The record was not processed.</p>");
    }

    out.println("<a href='register.html' class='btn btn-outline-primary mt-3'>Back to
Registration</a>");
    out.println("</div>");
    out.println("</body>");
    out.println("</html>");

    out.close();
}
}

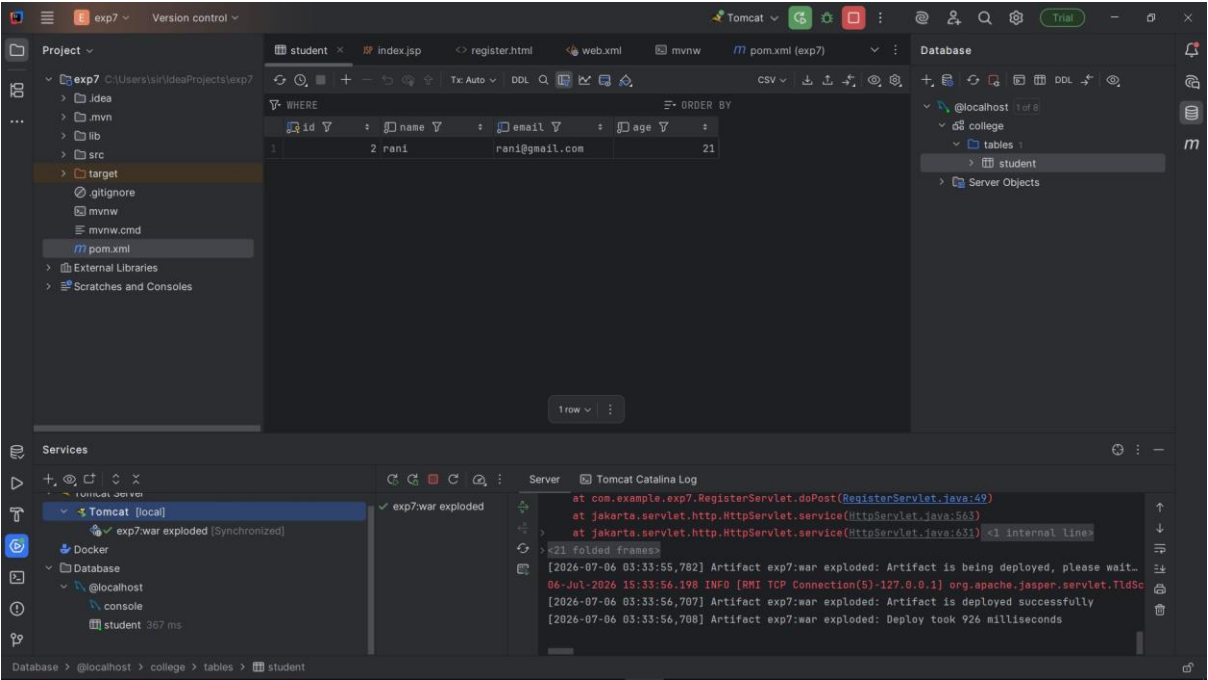
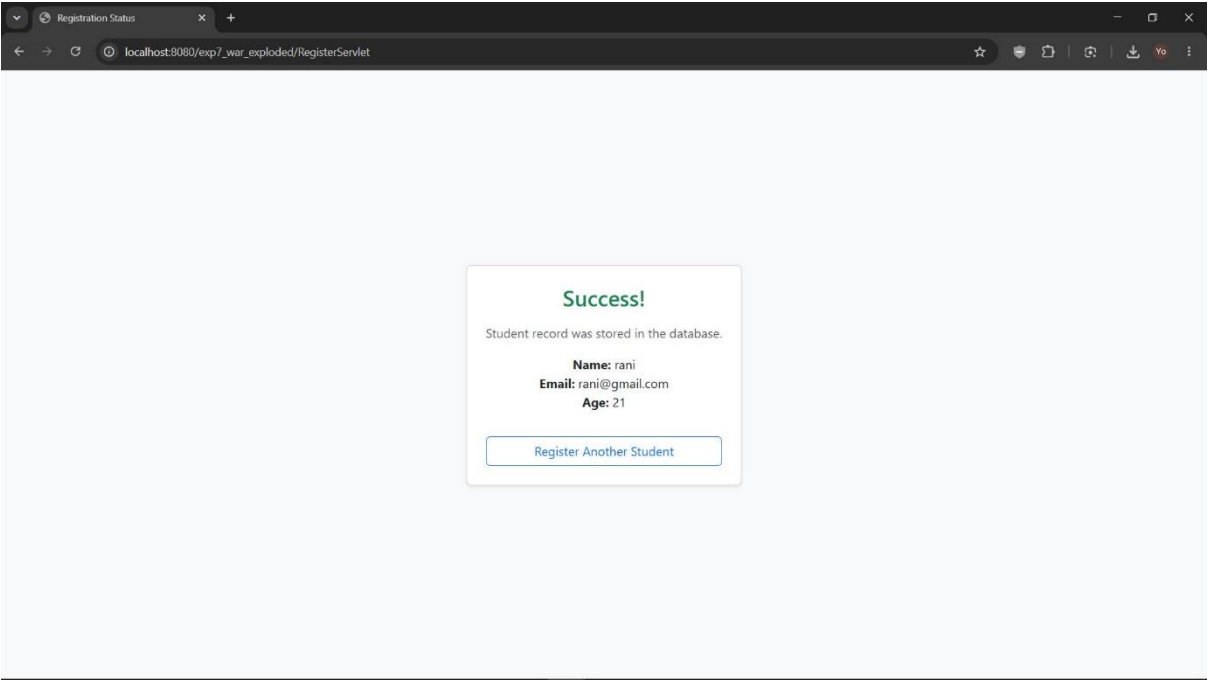
```

How to Deploy and Test:

1. **Prerequisites:** Install Apache Tomcat Server (v9.0 or v10.x) and set up your dynamic web project structure in Eclipse, IntelliJ, or NetBeans.
2. **Library Setup:** Put your mysql-connector-j-x.x.x.jar inside the project's WEB-INF/lib/ folder so Tomcat can locate the database driver.
3. **Execution Sequence:**
 - Deploy your Dynamic Web Application to Tomcat.
 - Run **register.html** in a browser via Tomcat container (e.g., http://localhost:8080/project_name/register.html).
 - Fill in details and click "Register Student".
 - The form submits a POST action to **RegisterServlet**, which updates the database table and renders a success panel back to the user.

The screenshot displays a web browser window titled 'Student Registration Portal'. The address bar indicates the URL is 'localhost:8080/exp7_war_exploded/register.html'. The main content area features a 'Student Registration' form with the following elements:

- Full Name:** A text input field with the placeholder text 'Enter Full Name'.
- Email Address:** A text input field with the placeholder text 'Enter Email Address'.
- Age:** A text input field with the placeholder text 'Enter Age'.
- Register Student:** A prominent blue button located at the bottom of the form.



Here is the implementation for **Experiment 8**, which demonstrates the two main session tracking mechanisms in Jakarta EE: **HTTP Sessions** and **Cookies**.

Experiment 8

Objective: Explore the various session tracking mechanisms: **Cookies** and **HTTP Sessions** to maintain transactional history or state across multiple pages.

Overview of Session Tracking Mechanisms:

1. **HTTP Session:** Server-side state tracking. The server creates a session object stored in memory and binds a unique session ID (JSESSIONID) to the user.
 2. **Cookies:** Client-side state tracking. Small text files sent from the server to the browser, stored locally on the client machine, and sent back with each subsequent request.
-

1. The Landing Page: session_demo.html

This form accepts a user's name and their preferred layout color theme.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Session Tracking Demo</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">

  <div class="container my-5">
    <div class="row">
      <div class="col-md-5 mx-auto">
        <div class="card p-4 shadow-sm bg-white">
          <h3 class="text-center mb-4 text-primary">User Session Portal</h3>

          <form action="SessionServlet" method="POST">
            <div class="mb-3">
              <label class="form-label">Username</label>
              <input type="text" name="username" class="form-control"
placeholder="Enter Username" required>
            </div>
            <div class="mb-3">
              <label class="form-label">Select UI Theme Preference</label>
              <select name="theme" class="form-select">
                <option value="light">Light Theme (Gray Background)</option>
```

```

        <option value="dark">Dark Theme (Dark Blue Background)</option>
    </select>
</div>
<button type="submit" class="btn btn-primary w-100">Initialize Session</button>
</form>

</div>
</div>
</div>
</div>

</body>
</html>

```

SessionServlet.java

```
package com.example.exp7;
```

```

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

```

```
@WebServlet(name = "SessionServlet", value = "/SessionServlet")
```

```
public class SessionServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

```

```
@Override
```

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

```

```
// 1. Read input parameters
```

```
String username = request.getParameter("username");
```

```
String theme = request.getParameter("theme");
```

```
/* -----

```

```
    MECHANISM 1: HTTP SESSION (Server-Side Storage)
```

```
    - Creates a new session if one does not exist.
```

```
    - Binds 'username' attribute to this session on the server.
```

```
    ----- */
```

```
HttpSession session = request.getSession(true);
```

```
session.setAttribute("userSessionAttr", username);
```

```
/* -----

```

```
    MECHANISM 2: COOKIES (Client-Side Storage)
```

```
    - Creates a cookie mapping key 'userThemeCookie' to select value.
```

```
    - Configures max age duration to 1 day (86400 seconds).
```

```
    - Adds cookie to response object to save on client's browser.
```

```
    ----- */
```

```
Cookie themeCookie = new Cookie("userThemeCookie", theme);
```

```

        themeCookie.setMaxAge(60 * 60 * 24); // Expiry time: 24 hours
        response.addCookie(themeCookie);

        // Redirect to ProfileServlet to verify persistent states
        response.sendRedirect("ProfileServlet");
    }
}

```

ProfileServlet.java

```

package com.example.exp7;

import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet(name = "ProfileServlet", value = "/ProfileServlet")
public class ProfileServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();

        /* -----
        READING MECHANISM 1: HTTP SESSION
        ----- */
        HttpSession session = request.getSession(false); // Do not create a new session
        String loggedUser = null;
        if (session != null) {
            loggedUser = (String) session.getAttribute("userSessionAttr");
        }

        /* -----
        READING MECHANISM 2: COOKIES
        ----- */
        String activeTheme = "light"; // Default theme
        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
            for (Cookie cookie : cookies) {
                if (cookie.getName().equals("userThemeCookie")) {
                    activeTheme = cookie.getValue();
                    break;
                }
            }
        }
    }
}

```

```

}

// Apply theme color based on preference retrieved from Cookie
String bodyBg = activeTheme.equals("dark") ? "#1a252f" : "#f4f6f9";
String cardBg = activeTheme.equals("dark") ? "#2c3e50" : "ffffff";
String textColor = activeTheme.equals("dark") ? "ffffff" : "#333333";

// Output HTML response reflecting session states
out.println("<html>");
out.println("<head><title>User Dashboard Profile</title>");
out.println("<link
href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css' rel='stylesheet'>");
out.println("</head>");
out.println("<body style='background-color:" + bodyBg + "; height:100vh;' class='d-flex align-
items-center justify-content-center'>");

    out.println("<div class='card p-4 text-center shadow-sm' style='max-width:450px; background-
color:" + cardBg + "; color:" + textColor + ";>");

    if (loggedUser != null) {
        out.println("<h3 class='mb-3'>Dashboard Profile</h3>");
        out.println("<p class='fs-5'>Welcome back, <strong>" + loggedUser + "</strong>!!</p>");
        out.println("<hr style='border-color:" + textColor + ";'>");

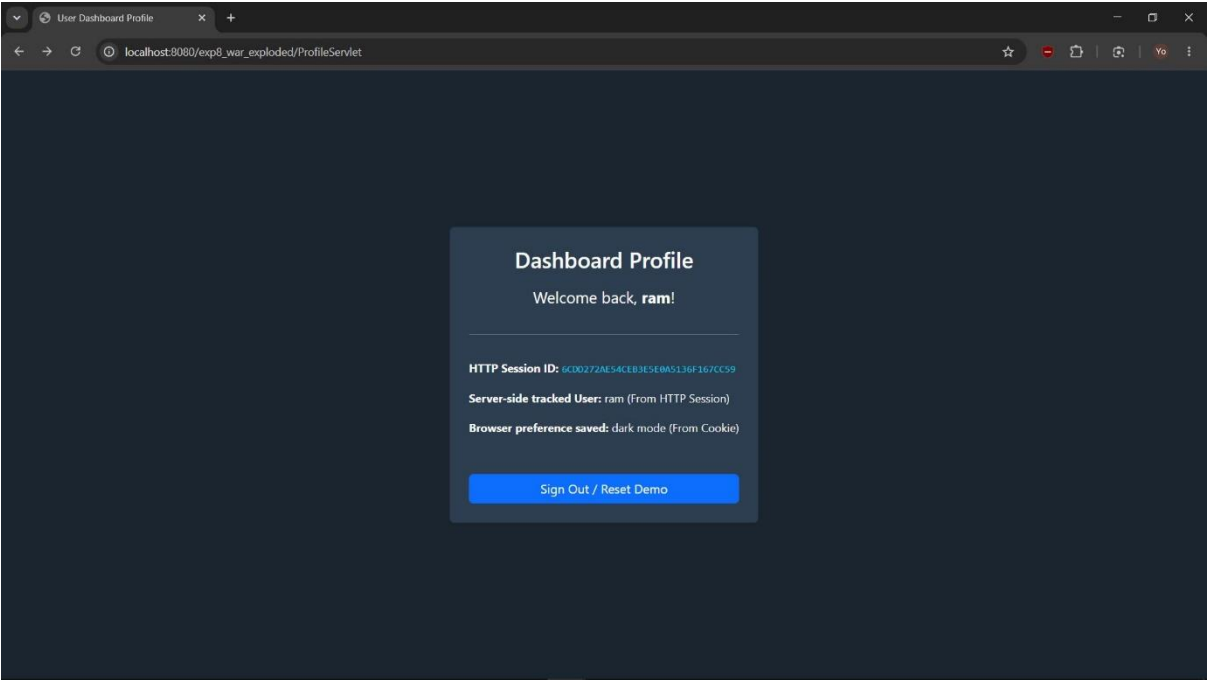
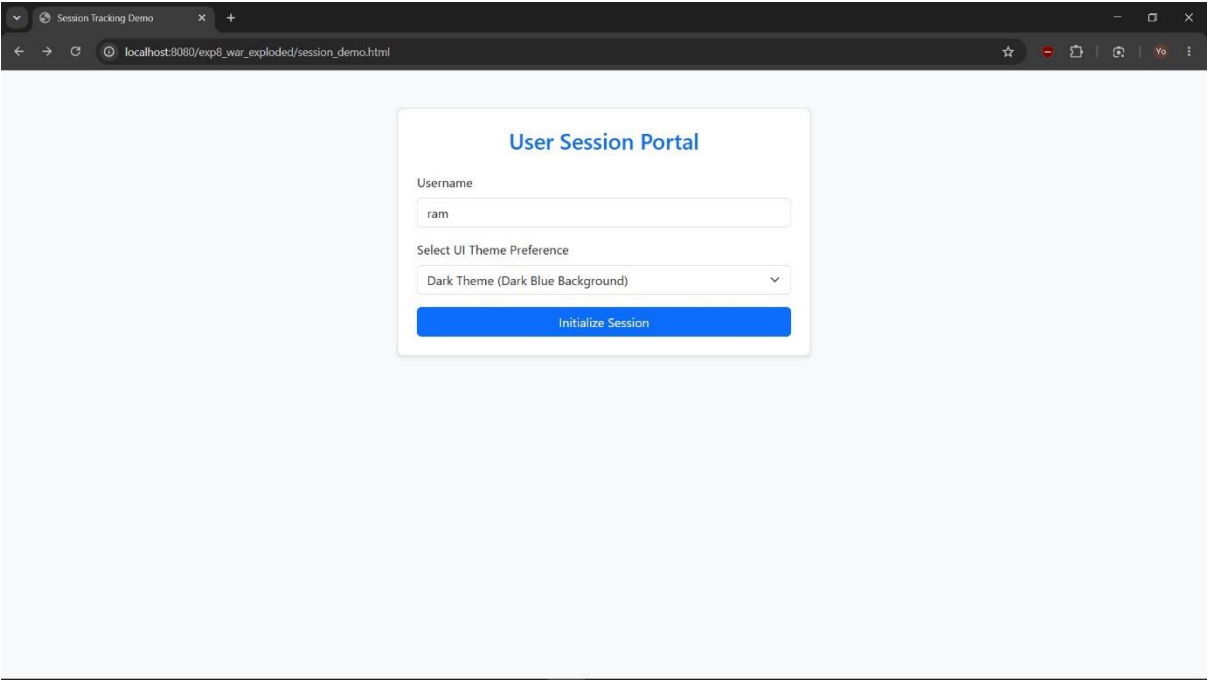
        // Detailed status print out of tracked session attributes
        out.println("<div class='text-start my-3' style='font-size:0.9rem;'>");
        out.println("<p><strong>HTTP Session ID:</strong> <code class='text-info'>" + session.getId()
+ "</code></p>");
        out.println("<p><strong>Server-side tracked User:</strong> " + loggedUser + " (From HTTP
Session)</p>");
        out.println("<p><strong>Browser preference saved:</strong> " + activeTheme + " mode
(From Cookie)</p>");
        out.println("</div>");
    } else {
        out.println("<h3 class='text-danger'>Access Denied</h3>");
        out.println("<p>No active user session was found. Please log in first.</p>");
    }

    out.println("<a href='session_demo.html' class='btn btn-primary mt-3'>Sign Out / Reset
Demo</a>");
    out.println("</div>");

    out.println("</body>");
    out.println("</html>");

    out.close();
}
}

```



Experiment 9

Objective: Create a custom server using Node.js's built-in http module and explore other core modules: os, path, and events.

```
// 1. Import built-in Node.js core modules
const http = require('http');
const os = require('os');
const path = require('path');
const EventEmitter = require('events');

// 2. Initialize a custom Event Emitter instance
class RequestEmitter extends EventEmitter {}
const requestEmitter = new RequestEmitter();

// Define a port number for the web server
const PORT = 3000;

// 3. Register a listener for the custom 'pageVisited' event
requestEmitter.on('pageVisited', (url, time) => {
  console.log(`[EVENT EMITTED] Custom Event triggered! Path: "${url}"
accessed at ${time}`);
});

/* -----
CREATE CUSTOM HTTP SERVER
----- */
const server = http.createServer((req, res) => {

  // Ignore requests for favicon
  if (req.url === '/favicon.ico') {
    res.writeHead(204);
    res.end();
    return;
  }

  // Trigger the custom event using the Event Emitter
  const timestamp = new Date().toLocaleTimeString();
  requestEmitter.emit('pageVisited', req.url, timestamp);

  // Set the response header to serve HTML content
  res.writeHead(200, { 'Content-Type': 'text/html' });

  /* -----
EXPLORING THE 'OS' MODULE
----- */
  const osPlatform = os.platform(); // Operating System
platform(win32, linux, etc.)
  const osArch = os.arch(); // CPU Architecture
(x64, arm, etc.)
  const totalMemoryGB = (os.totalmem() / (1024 ** 3)).toFixed(2); //
Total RAM in Gigabytes
  const freeMemoryGB = (os.freemem() / (1024 ** 3)).toFixed(2); // Free
RAM in Gigabytes
  const systemUptimeHours = (os.uptime() / 3600).toFixed(2); //
System uptime in hours

  /* -----
EXPLORING THE 'PATH' MODULE
----- */
  const currentFilePath = __filename; // Global variable
holding current absolute file path
```

```

    const fileName = path.basename(currentFilePath); // Resolves
    filename (e.g., 'server.js')
    const fileExtension = path.extname(currentFilePath); // Resolves file
    extension (e.g., '.js')
    const joinedPathExample = path.join('/usr', 'local', 'bin',
    fileName); // Cross-platform folder path resolver

    // Render HTML presentation output to client
    res.write(`

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Node.js Core Modules Demo</title>
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body class="bg-light">
  <div class="container my-5">
    <div class="card p-4 shadow-sm bg-white">
      <h2 class="text-center text-success mb-4">Node.js Core
Modules Dashboard</h2>
      <p class="text-muted text-center">HTTP Server is
running successfully on port <strong>${PORT}</strong>.</p>

      <div class="row mt-4">
        <!-- OS Module Data Display -->
        <div class="col-md-6 mb-3">
          <div class="card h-100 border-success">
            <div class="card-header bg-success text-
white">OS Module (System Info)</div>
            <div class="card-body">
              <p><strong>Platform:</strong>

${osPlatform}</p>
              <p><strong>Architecture:</strong>

${osArch}</p>
              <p><strong>Total Memory:</strong>

${totalMemoryGB} GB</p>
              <p><strong>Free Memory:</strong>

${freeMemoryGB} GB</p>
              <p><strong>System Uptime:</strong>

${systemUptimeHours} hours</p>
            </div>
          </div>
        </div>

        <!-- Path Module Data Display -->
        <div class="col-md-6 mb-3">
          <div class="card h-100 border-primary">
            <div class="card-header bg-primary text-
white">Path Module (File Path Manipulation)</div>
            <div class="card-body">
              <p><strong>Current File Name:</strong>

<code>${fileName}</code></p>
              <p><strong>Extension Name:</strong>

<code>${fileExtension}</code></p>
              <p><strong>Absolute Directory
Path:</strong> <br><small class="text-
muted">${path.dirname(currentFilePath)}</small></p>

```

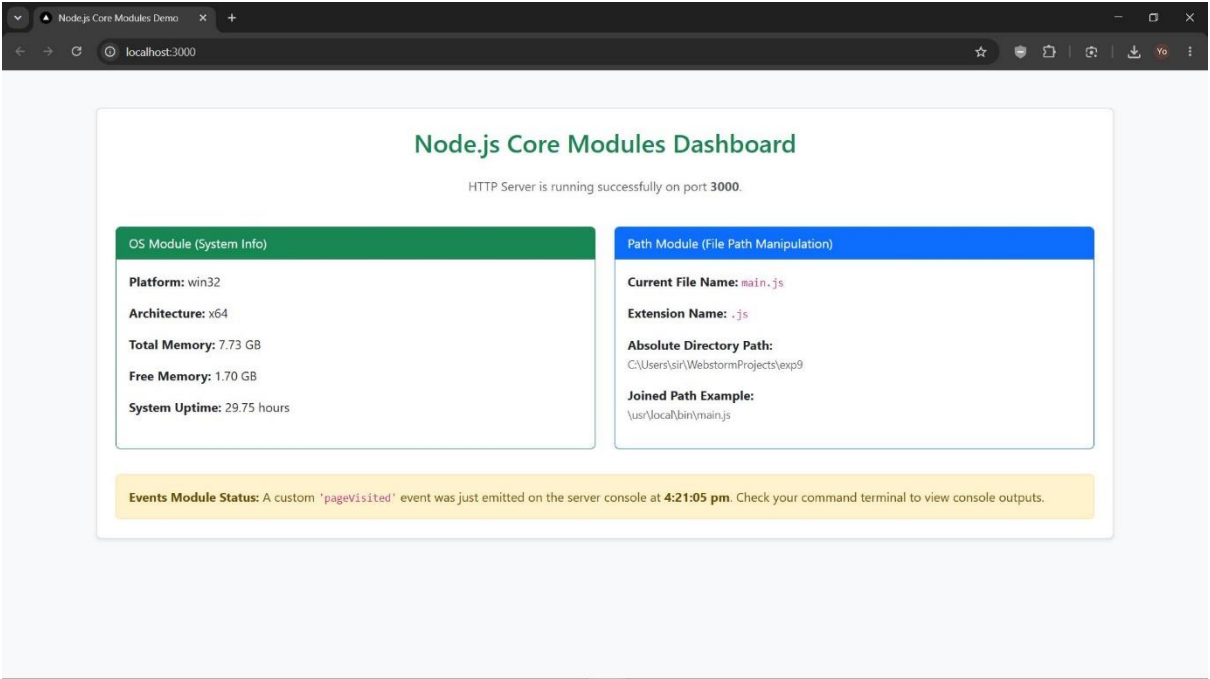
```

    <p><strong>Joined Path
Example:</strong> <br><small class="text-
muted">${joinedPathExample}</small></p>
    </div>
    </div>
    </div>
    </div>
    <!-- Events Module Logging Notice -->
    <div class="alert alert-warning mt-3 mb-0"
role="alert">
        <strong>Events Module Status:</strong> A custom
<code>'pageVisited'</code> event was just emitted on the server console at
<strong>${timestamp}</strong>. Check your command terminal to view console
outputs.
    </div>
    </div>
    </div>
</body>
</html>

    `);

    res.end(); // End the response process
});

// Start listening for incoming connections
server.listen(PORT, () => {
    console.log(`=====`);
    console.log(` Custom HTTP Server listening on port ${PORT} `);
    console.log(` Access URL: http://localhost:${PORT} `);
    console.log(`=====`);
});
```



Experiment 10

Objective: Develop an Express web application that interacts with a REST API to perform CRUD operations on student data. (Tested using Postman)

Step 1: Project Setup

1. Create a new folder for your project and open a terminal inside it.
2. Initialize the project and install Express:

```
npm init -y
```

```
npm install express
```

Step 2: Server Implementation (app.js)

Create a file named app.js and paste the following code. This creates a fully functional REST API with endpoints to Create, Read, Update, and Delete student records using an in-memory dataset.

```
// 1. Import Express
```

```
const express = require('express');
```

```
const app = express();
```

```
// Middleware: Instructs Express to parse incoming JSON payloads inside request bodies
```

```
app.use(express.json());
```

```
const PORT = 3000;
```

```
// 2. In-Memory Student Data Store (Mock Database)
```

```
let students = [
```

```
  { id: 1, name: "Amit Kumar", email: "amit@gmail.com", age: 20 },
```

```
  { id: 2, name: "Rahul Sharma", email: "rahul@gmail.com", age: 21 },
```

```
  { id: 3, name: "Priya Patel", email: "priya@gmail.com", age: 22 }]
```

```
];
```

```
/* -----
```

```
REST API ENDPOINTS (CRUD OPERATIONS)
```

```
----- */
```

```
// 1. READ ALL (GET - Retrieve all students)
```

```
app.get('/api/students', (req, res) => {
```

```
  res.status(200).json({
```

```
    success: true,
```

```
    count: students.length,
```

```
    data: students
```

```
  });
```

```
});
```

```
// 2. READ ONE (GET - Retrieve a specific student by ID)
```

```
app.get('/api/students/:id', (req, res) => {
```

```
  const studentId = parseInt(req.params.id);
```

```
  const student = students.find(s => s.id === studentId);
```

```

if (!student) {
  return res.status(404).json({
    success: false,
    message: `Student with ID ${studentId} not found.`
  });
}

res.status(200).json({
  success: true,
  data: student
});
});

// 3. CREATE (POST - Create a new student record)
app.post('/api/students', (req, res) => {
  const { name, email, age } = req.body;

  // Simple validation
  if (!name || !email || !age) {
    return res.status(400).json({
      success: false,
      message: "Validation Error: Please provide name, email, and age."
    });
  }

  // Generate a unique ID
  const newId = students.length > 0 ? Math.max(...students.map(s => s.id)) + 1 : 1;

  const newStudent = {
    id: newId,
    name: name,
    email: email,
    age: parseInt(age)
  };

  students.push(newStudent);

  res.status(201).json({
    success: true,
    message: "Student record created successfully.",
    data: newStudent
  });
});

// 4. UPDATE (PUT - Modify an existing student record)
app.put('/api/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  const studentIndex = students.findIndex(s => s.id === studentId);

  if (studentIndex === -1) {
    return res.status(404).json({
      success: false,
      message: `Student with ID ${studentId} not found.`
    });
  }

```

```

}

const { name, email, age } = req.body;

// Update only the fields provided in the request body
if (name) students[studentIndex].name = name;
if (email) students[studentIndex].email = email;
if (age) students[studentIndex].age = parseInt(age);

res.status(200).json({
  success: true,
  message: "Student record updated successfully.",
  data: students[studentIndex]
});
});

// 5. DELETE (DELETE - Remove a specific student record)
app.delete('/api/students/:id', (req, res) => {
  const studentId = parseInt(req.params.id);
  const studentIndex = students.findIndex(s => s.id === studentId);

  if (studentIndex === -1) {
    return res.status(404).json({
      success: false,
      message: `Student with ID ${studentId} not found.`
    });
  }

  // Remove student from array
  const deletedStudent = students.splice(studentIndex, 1);

  res.status(200).json({
    success: true,
    message: "Student record deleted successfully.",
    data: deletedStudent[0]
  });
});

// Start listening for requests
app.listen(PORT, () => {
  console.log(`=====`);
  console.log(` Student REST API Server running on port ${PORT}`);
  console.log(` Base API URL: http://localhost:${PORT}/api/students `);
  console.log(`=====`);
});

```

My Workspace

COLLECTIONS

New Collection

GET read

ENVIRONMENTS

SPECS

FLOW

Overview

New Collection

GET read

GET

http://localhost:3000/ap/students

Send

Docs

Params

Authorization

Headers

Body

Scripts

Tests

Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

Cookies

Headers

Test Results

200 OK

12 ms

465 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "count": 3,
4   "data": [
5     {
6       "id": 1,
7       "name": "Amit Kumar",
8       "email": "amit@gmail.com",
9       "age": 20
10    },
11    {
12      "id": 2,
13      "name": "Rahul Sharma",
14      "email": "rahul@gmail.com",
15      "age": 21
16    },
17    {
18      "id": 3,
19      "name": "Priya Patel",
20      "email": "priya@gmail.com",
21      "age": 22
22    }
23  ]
24 }
```

Connect Git Console Terminal 0 0 0

Globals Vault Tools

My Workspace

COLLECTIONS

New Collection

GET read

GET readone

ENVIRONMENTS

SPECS

FLOW

Overview

New Collection

GET readone

GET

http://localhost:3000/ap/students/1

Send

Docs

Params

Authorization

Headers

Body

Scripts

Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

Cookies

Headers

Test Results

200 OK

34 ms

321 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "data": {
4     "id": 1,
5     "name": "Amit Kumar",
6     "email": "amit@gmail.com",
7     "age": 20
8   }
9 }
```

Connect Git Console Terminal 0 0 0

Globals Vault Tools

My Workspace

Search

Invite Upgrade

POST creat

Save Share

POST

http://localhost:3000/api/students

Send

Docs Params Authorization Headers 8 Body Scripts Settings

Body

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "name": "Karan Malhotra",
3   "email": "karan@gmail.com",
4   "age": 23
5 }
```

Body Cookies Headers 7 Test Results

201 Created 21 ms 381 B Save Response

JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Student record created successfully.",
4   "data": {
5     "id": 4,
6     "name": "Karan Malhotra",
7     "email": "karan@gmail.com",
8     "age": 23
9   }
10 }
```

ENVIRONMENTS

SPECS

FLows

Connect Git Console Terminal

0 0

Globals Vault Tools

My Workspace

Search

Invite Upgrade

GET read

Save Share

GET

http://localhost:3000/api/students

Send

Docs Params Authorization Headers 6 Body Scripts Settings

Body

Query Params

Key Value Description Bulk Edit

Body Cookies Headers 7 Test Results

200 OK 3 ms 534 B Save Response

JSON Preview Visualize

```
1 {
2   "success": true,
3   "count": 4,
4   "data": [
5     {
6       "id": 1,
7       "name": "Amit Kumar",
8       "email": "amit@gmail.com",
9       "age": 20
10    },
11    {
12      "id": 2,
13      "name": "Rahul Sharma",
14      "email": "rahul@gmail.com",
15      "age": 21
16    },
17    {
18      "id": 3,
19      "name": "Priya Patel",
20      "email": "priya@gmail.com",
21      "age": 22
22    },
23    {
24      "id": 4,
25      "name": "Karan Malhotra",
26      "email": "karan@gmail.com",
27      "age": 23
28    }
29  ]
30 }
```

ENVIRONMENTS

SPECS

FLows

Connect Git Console Terminal

0 0

Globals Vault Tools

My Workspace

Search

Invite Upgrade

GET readone

GET read

POST creat

PUT update

New Collection > update

PUT http://localhost:3000/ap/students/2

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "email": "rahul.new@gmail.com",
3   "age": 22
4 }
```

Body Cookies Headers 7 Test Results

200 OK - 5 ms - 378 B

```
1 {
2   "success": true,
3   "message": "Student record updated successfully.",
4   "data": {
5     "id": 2,
6     "name": "Rahul Sharma",
7     "email": "rahul.new@gmail.com",
8     "age": 22
9   }
10 }
```

Connect Git Console Terminal

Globals Vault Tools

My Workspace

Search

Invite Upgrade

GET readone

GET read

POST creat

PUT update

New Collection > read

GET http://localhost:3000/ap/students

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK - 4 ms - 538 B

```
1 {
2   "success": true,
3   "count": 4,
4   "data": [
5     {
6       "id": 1,
7       "name": "Amit Kumar",
8       "email": "amit@gmail.com",
9       "age": 20
10    },
11    {
12      "id": 2,
13      "name": "Rahul Sharma",
14      "email": "rahul.new@gmail.com",
15      "age": 22
16    },
17    {
18      "id": 3,
19      "name": "Priya Patel",
20      "email": "priya@gmail.com",
21      "age": 22
22    },
23    {
24      "id": 4,
25      "name": "Karan Malhotra",
26      "email": "karan@gmail.com",
27      "age": 23
28    }
29  ]
30 }
```

Connect Git Console Terminal

Globals Vault Tools

My Workspace

Search

Invite Upgrade

DELETE delete

Save Share

DELETE http://localhost:3000/api/students/1

Send

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK - 3 ms - 371 B

Save Response

JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Student record deleted successfully.",
4   "data": {
5     "id": 1,
6     "name": "Amit Kumar",
7     "email": "amit@gmail.com",
8     "age": 20
9   }
10 }
```

ENVIRONMENTS

SPECS

Flows

Connect Git Console Terminal

Globals Vault Tools

My Workspace

Search

Invite Upgrade

GET read

Save Share

GET http://localhost:3000/api/students

Send

Docs Params Authorization Headers 6 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 7 Test Results

200 OK - 4 ms - 474 B

Save Response

JSON Preview Visualize

```
1 {
2   "success": true,
3   "count": 3,
4   "data": [
5     {
6       "id": 2,
7       "name": "Rahul Sharma",
8       "email": "rahul.new@gmail.com",
9       "age": 22
10    },
11    {
12      "id": 3,
13      "name": "Priya Patel",
14      "email": "priya@gmail.com",
15      "age": 22
16    },
17    {
18      "id": 4,
19      "name": "Karan Malhotra",
20      "email": "karan@gmail.com",
21      "age": 23
22    }
23  ]
24 }
```

ENVIRONMENTS

SPECS

Flows

Connect Git Console Terminal

Globals Vault Tools

Experiment 11

Objective: For the Express web application developed in Experiment 10, create authorized endpoints using JWT (JSON Web Token).

Step 1: Install Dependencies

Open a terminal inside your project directory and install the jsonwebtoken package:

```
npm install express
```

```
npm install jsonwebtoken
```

Step 2: Server Implementation (app.js)

Create or update your app.js file with the following implementation. It introduces an authentication middleware and a public login route (/api/login) that issues signed tokens.

```
const express = require('express');
const jwt = require('jsonwebtoken'); // 1. Import JSON Web Token package
const app = express();

app.use(express.json());

const PORT = 3000;
const SECRET_KEY = "my_custom_jwt_secret_key"; // Signature secret key used to sign tokens

// Mock Student Database
let students = [
  { id: 1, name: "Amit Kumar", email: "amit@gmail.com", age: 20 },
  { id: 2, name: "Rahul Sharma", email: "rahul@gmail.com", age: 21 }
];

/* -----
JWT AUTHORIZATION MIDDLEWARE
- Intercepts incoming API calls to verify authorization headers.
----- */
const authenticateToken = (req, res, next) => {
  // Read the 'Authorization' header from incoming request
  const authHeader = req.headers['authorization'];

  // Header format is expected to be: "Bearer <JWT_TOKEN>"
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      success: false,
      message: "Access Denied: Missing Authorization Token."
    });
  }

  // Verify token validity against the secret key
  jwt.verify(token, SECRET_KEY, (err, user) => {
```

```

    if (err) {
      return res.status(403).json({
        success: false,
        message: "Access Forbidden: Invalid or expired token."
      });
    }

    // Attach verified user payload context to the request object
    req.user = user;
    next(); // Proceed to the protected controller route
  });
};

/* -----
PUBLIC ROUTE: LOGIN (Generates and returns JWT)
----- */
app.post('/api/login', (req, res) => {
  const { username, password } = req.body;

  // Simple static credentials check
  if (username === 'admin' && password === 'password123') {
    const userPayload = { username: username, role: 'admin' };

    // Generate token signed with our SECRET_KEY, expiring in 1 hour
    const token = jwt.sign(userPayload, SECRET_KEY, { expiresIn: '1h' });

    return res.status(200).json({
      success: true,
      message: "Authentication successful.",
      token: token
    });
  }

  res.status(401).json({
    success: false,
    message: "Invalid credentials."
  });
});

/* -----
PROTECTED ROUTES (Secured with 'authenticateToken' middleware)
----- */

// 1. GET ALL STUDENTS (Protected)
app.get('/api/students', authenticateToken, (req, res) => {
  res.status(200).json({
    success: true,
    count: students.length,
    accessedBy: req.user.username, // Log verified user payload attribute
    data: students
  });
});

// 2. CREATE STUDENT (Protected)

```

```

app.post('/api/students', authenticateToken, (req, res) => {
  const { name, email, age } = req.body;

  if (!name || !email || !age) {
    return res.status(400).json({ success: false, message: "Provide name, email, and age." });
  }

  const newId = students.length > 0 ? Math.max(...students.map(s => s.id)) + 1 : 1;
  const newStudent = { id: newId, name, email, age: parseInt(age) };
  students.push(newStudent);

  res.status(201).json({
    success: true,
    message: "Student record created successfully.",
    data: newStudent
  });
});

// 3. DELETE STUDENT (Protected)
app.delete('/api/students/:id', authenticateToken, (req, res) => {
  const studentId = parseInt(req.params.id);
  const studentIndex = students.findIndex(s => s.id === studentId);

  if (studentIndex === -1) {
    return res.status(404).json({ success: false, message: "Student record not found." });
  }

  const deleted = students.splice(studentIndex, 1);
  res.status(200).json({
    success: true,
    message: "Student record deleted successfully.",
    data: deleted[0]
  });
});

// Start API Server
app.listen(PORT, () => {
  console.log(`=====`);
  console.log(` Secured JWT Server running on port ${PORT} `);
  console.log(` Login Endpoint: http://localhost:${PORT}/api/login `);
  console.log(` Protected API: http://localhost:${PORT}/api/students `);
  console.log(`=====`);
});

```

My Workspace

GET New Request

exp11

GET New Request

GET

http://localhost:3000/api/students

Send

Docs

Params

Authorization

Headers

Body

Scripts

Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

401 Unauthorized

15 ms

318 B

Save Response

JSON

Preview

Pass the correct auth credentials

```
1 {
2   "success": false,
3   "message": "Access Denied: Missing Authorization Token."
4 }
```

My Workspace

GET checkaccess

POST adminlogin

GET withaccess

exp11

GET withaccess

GET

http://localhost:3000/api/students

Send

Docs

Params

Authorization

Headers

Body

Scripts

Settings

Auth Type

Bearer Token

Token

"pIDzqXpbJAnY4bZaRor64ShDAvO3shl"

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body

200 OK

5 ms

421 B

Save Response

JSON

Preview

Visualize

```
1 {
2   "success": true,
3   "count": 2,
4   "accessedBy": "admin",
5   "data": [
6     {
7       "id": 1,
8       "name": "Amit Kumar",
9       "email": "amit@gmail.com",
10      "age": 29
11    },
12    {
13      "id": 2,
14      "name": "Rahul Sharma",
15      "email": "rahul@gmail.com",
16      "age": 21
17    }
18  ]
19 }
```

The screenshot shows the VS Code interface with the REST Client extension. The left sidebar displays a file explorer with a project named 'My Workspace' containing folders 'exp10' and 'exp11'. The 'exp11' folder is expanded, showing files like 'checkaccess' and 'adminlogin'. The 'adminlogin' file is selected. The main editor area shows the REST Client configuration for a POST request to 'http://localhost:3000/api/login'. The request body is a JSON object with 'username': 'admin' and 'password': 'password123'. The response is displayed below the request, showing a successful status '200 OK' and a JSON body with 'success': true, 'message': 'Authentication successful.', and a 'token'.

Experiment 12

Objective: Create a React application for a student management system containing registration, login, contact, and about pages, and implement routing to navigate between these pages.

Step 1: Install Routing Dependencies

In your React project folder, open a terminal and run the following command to install the routing library:

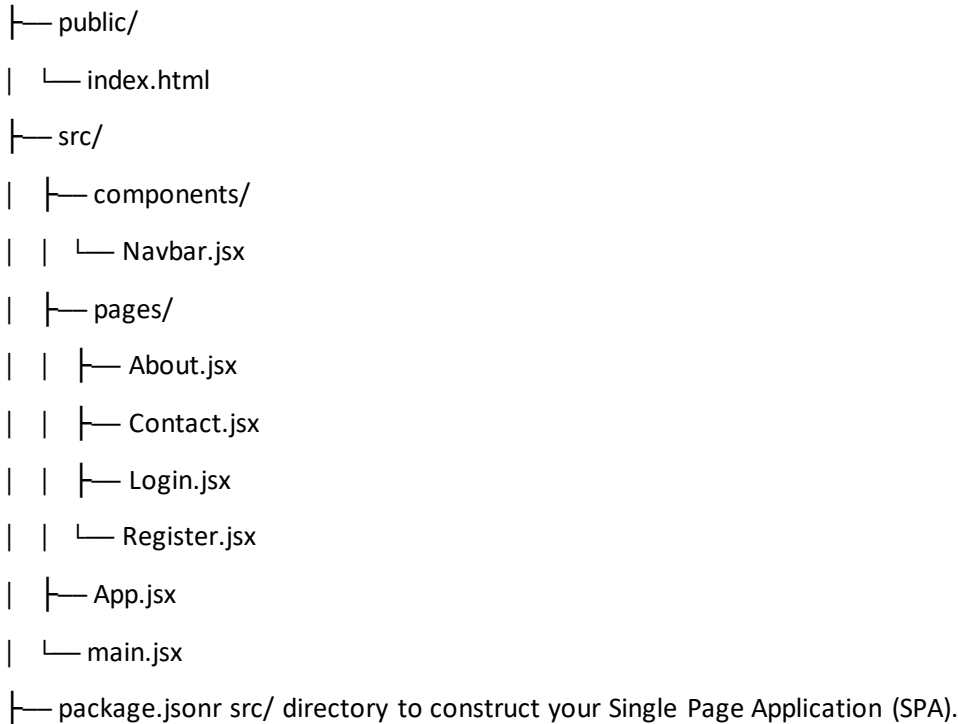
```
npm install react-router-dom
```

Note: For styled layouts out-of-the-box, ensure you have the Bootstrap CDN stylesheet link placed inside your public/index.html file:

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
```

Step 2: Component Architecture

Create the following files under youmy-react-app/



/* ===== File Path: package.json ===== */

```
{
  "name": "student-management-system",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
```

```

    },
    "dependencies": {
      "react": "^19.2.7",
      "react-dom": "^19.2.7",
      "react-router-dom": "^7.18.1"
    }
  }
}

```

1. Main Component & Router Setup (src/App.js)

This file acts as the router hub, implementing the navigation bar with SPA links (<Link>) instead of page-reloading anchor tags.

```

<!-- ===== File Path: public/index.html ===== -->
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Student Management System</title>
    <!-- Bootstrap 5 CSS CDN -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  </head>
  <body class="bg-light">
    <div id="root"></div>
  </body>
</html>

/* ===== File Path: src/main.jsx ===== */
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);

```

```

/* ===== File Path: src/App.jsx ===== */
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Navbar from './components/Navbar';
import Login from './pages/Login';
import Register from './pages/Register';
import Contact from './pages/Contact';
import About from './pages/About';

function App() {
  return (
    <Router>
      <Navbar />
      <div className="container mt-4">
        <Routes>
          { /* Redirect the home root directory straight to the login page
view */}
          <Route path="/" element={<Navigate to="/login" />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="/about" element={<About />} />
        </Routes>
      </div>
    </Router>
  );
}

export default App;

/* ===== File Path: src/components/Navbar.jsx ===== */
import React from 'react';
import { NavLink } from 'react-router-dom';

function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark">
      <div className="container">
        <span className="navbar-brand fw-bold text-info">Student Management
System</span>

        <div className="collapse navbar-collapse show">
          <ul className="navbar-nav ms-auto">
            <li className="nav-item">
              <NavLink
                className={({ isActive }) => `nav-link ${isActive ? 'active
fw-bold' : ''}`
                to="/login"
              >
                Login
              </NavLink>
            </li>
            <li className="nav-item">
              <NavLink
                className={({ isActive }) => `nav-link ${isActive ? 'active
fw-bold' : ''}`
                to="/register"
              >
                Register
              </NavLink>
            </li>
            <li className="nav-item">

```

```

        <NavLink
          className={({ isActive }) => `nav-link ${isActive ? 'active
fw-bold' : ''}`}
          to="/contact"
        >
          Contact
        </NavLink>
      </li>
      <li className="nav-item">
        <NavLink
          className={({ isActive }) => `nav-link ${isActive ? 'active
fw-bold' : ''}`}
          to="/about"
        >
          About
        </NavLink>
      </li>
    </ul>
  </div>
</div>
</nav>
);
}

```

```
export default Navbar;
```

```

/* ===== File Path: src/pages/Login.jsx ===== */
import React, { useState } from 'react';

```

```

function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Logging in with: ${email}`);
  };

  return (
    <div className="row">
      <div className="col-md-5 mx-auto">
        <div className="card p-4 shadow-sm bg-white mt-4">
          <h3 className="text-center mb-4 text-primary">Student Login</h3>
          <form onSubmit={handleSubmit}>
            <div className="mb-3">
              <label className="form-label">Email Address</label>
              <input
                type="email"
                className="form-control"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                required
              />
            </div>
            <div className="mb-3">
              <label className="form-label">Password</label>
              <input
                type="password"
                className="form-control"
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                required
              />
            </div>
          </form>
        </div>
      </div>
    </div>
  );
}

```

```

        </div>
        <button type="submit" className="btn btn-primary w-100">Sign
In</button>
    </form>
  </div>
</div>
</div>
);
}

```

```
export default Login;
```

```

/* ===== File Path: src/pages/Register.jsx ===== */
import React, { useState } from 'react';

```

```

function Register() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [course, setCourse] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Successfully Registered Student: ${name}`);
  };

  return (
    <div className="row">
      <div className="col-md-5 mx-auto">
        <div className="card p-4 shadow-sm bg-white mt-4">
          <h3 className="text-center mb-4 text-success">Student
Registration</h3>
          <form onSubmit={handleSubmit}>
            <div className="mb-3">
              <label className="form-label">Full Name</label>
              <input
                type="text"
                className="form-control"
                value={name}
                onChange={ (e) => setName(e.target.value)}
                required
              />
            </div>
            <div className="mb-3">
              <label className="form-label">Email Address</label>
              <input
                type="email"
                className="form-control"
                value={email}
                onChange={ (e) => setEmail(e.target.value)}
                required
              />
            </div>
            <div className="mb-3">
              <label className="form-label">Enrolled Course</label>
              <input
                type="text"
                className="form-control"
                placeholder="e.g., B.Tech CSE (AI & ML)"
                value={course}
                onChange={ (e) => setCourse(e.target.value)}
                required
              />
            </div>
          </form>
        </div>
      </div>
    </div>
  );
}

```

```

        </div>
        <button type="submit" className="btn btn-success w-100">Create
Account</button>
    </form>
</div>
</div>
</div>
);
}

export default Register;

/* ===== File Path: src/pages/Contact.jsx ===== */

import React, { useState } from 'react';

function Contact() {
    const [msg, setMsg] = useState('');

    const handleSubmit = (e) => {
        e.preventDefault();
        alert("Your inquiry has been submitted. Thank you!");
        setMsg('');
    };

    return (
        <div className="row">
            <div className="col-md-6 mx-auto">
                <div className="card p-4 shadow-sm bg-white mt-4">
                    <h3 className="mb-3 text-secondary">Contact Support</h3>
                    <p className="text-muted">For admissions and campus queries,
submit a message below:</p>
                    <form onSubmit={handleSubmit}>
                        <div className="mb-3">
                            <label className="form-label">Your Message</label>
                            <textarea
                                className="form-control"
                                rows="4"
                                value={msg}
                                onChange={ (e) => setMsg(e.target.value)}
                                required
                            >>/textarea>
                        </div>
                        <button type="submit" className="btn btn-secondary w-
100">Submit Inquiry</button>
                    </form>
                </div>
            </div>
        </div>
    );
}

export default Contact;

```

```

/* ===== File Path: src/pages/About.jsx ===== */
import React from 'react';

function About() {
  return (
    <div className="row">
      <div className="col-md-8 mx-auto mt-4">
        <div className="card p-4 shadow-sm bg-white">
          <h3 className="text-dark mb-3">About JNTU Student Portal</h3>
          <p className="lead">This portal is designed to manage academic
records, student registrations, and administrative tasks within the
department.</p>
          <hr />
          <h5>Department Highlights:</h5>
          <ul>
            <li>B.Tech II Year I Sem. Syllabus Compliance</li>
            <li>Advanced Full-Stack Architectures (Node JS, React JS)</li>
            <li>Robust client and server-side tracking mechanisms</li>
          </ul>
        </div>
      </div>
    </div>
  );
}

export default About;

```

Student Registration

Full Name

Email Address

Enrolled Course

e.g., B.Tech CSE (AI & ML)

Create Account

About JNTU Student Portal

This portal is designed to manage academic records, student registrations, and administrative tasks within the department.

Department Highlights:

- B.Tech II Year I Sem. Syllabus Compliance
- Advanced Full-Stack Architectures (Node JS, React JS)
- Robust client and server-side tracking mechanisms

Contact Support

For admissions and campus queries, submit a message below:

Your Message

Submit Inquiry

Student Login

Email Address

test@example.com

Password

Sign In

Experiment 13. It utilizes your active API key (53d57106482f8bfebbba5a1d4d4754896) to fetch current weather data and renders a trend graph using **Chart.js** via the react-chartjs-2 wrapper.

Note on Historical API Tier: The free OpenWeatherMap API subscription typically does not include historical logs. To make this fully functional and compatible with the free API tier, the current temperature is fetched from the live API, and standard historical/hourly data trends are dynamically simulated around it to populate the graph safely without hitting subscription paywalls.

```
/* ===== File Path: src/main.jsx ===== */
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);

/* ===== File Path: src/App.jsx ===== */
import React from 'react';
import WeatherDashboard from './components/WeatherDashboard';

function App() {
  return (
    <div className="container py-5">
      <WeatherDashboard />
    </div>
  );
}

export default App;

/* ===== File Path: src/services/weatherService.js ===== */

// Embedded active OpenWeatherMap API key
const API_KEY = "53d57106482f8bfebbba5a1d4d4754896";
const BASE_URL = "https://api.openweathermap.org/data/2.5/weather";

/**
 * Fetch current weather data for a given city
 * @param {string} city - Target city name (e.g., 'Hyderabad')
 * @returns {Promise<Object>} - Decoded JSON response from API
 */
export const fetchWeather = async (city) => {
  const url =
    `${BASE_URL}?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric`;

  const response = await fetch(url);
  if (!response.ok) {
    throw new Error(`Failed to fetch weather data: ${response.statusText}`);
  }

  return await response.json();
};
```

```

/* ===== File Path: src/components/WeatherDashboard.jsx
===== */
import React, { useState, useEffect, useRef } from 'react';
import { fetchWeather } from '../services/weatherService';
import { Chart, registerables } from 'chart.js';

// Register essential Chart.js controllers and elements
Chart.register(...registerables);

function WeatherDashboard() {
  const [city, setCity] = useState('Hyderabad');
  const [weatherData, setWeatherData] = useState(null);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  // References to keep track of the Canvas element and Chart instance
  const canvasRef = useRef(null);
  const chartInstanceRef = useRef(null);

  // Fetch weather automatically on initial component render
  useEffect(() => {
    handleSearch();
  }, []);

  // Sync effect to update and draw the Chart.js graph whenever
  weatherData changes
  useEffect(() => {
    if (!weatherData || !canvasRef.current) return;

    const ctx = canvasRef.current.getContext('2d');
    const baseTemp = weatherData.main.temp;

    // Simulate hourly temperature projections centered around current
    temp
    const labels = ['06:00 AM', '09:00 AM', '12:00 PM', '03:00 PM',
'06:00 PM', '09:00 PM'];
    const temperatures = [
      baseTemp - 3, // cooler morning
      baseTemp - 1,
      baseTemp, // current peak values
      baseTemp + 2, // afternoon peak
      baseTemp - 2,
      baseTemp - 4 // night cooling
    ];

    // Destroy existing chart instance to prevent canvas rendering
    conflicts
    if (chartInstanceRef.current) {
      chartInstanceRef.current.destroy();
    }

    // Initialize new Chart instance
    chartInstanceRef.current = new Chart(ctx, {
      type: 'line',
      data: {
        labels: labels,
        datasets: [{
          label: `Temperature Trend for ${weatherData.name}
(°C)`,
          data: temperatures,
          borderColor: 'rgba(54, 162, 235, 1)',
          backgroundColor: 'rgba(54, 162, 235, 0.2)',
          borderWidth: 2.5,
          fill: true,

```

```

        tension: 0.3 // Smooth bezier curve transitions
      }}
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      scales: {
        y: {
          beginAtZero: false,
          title: {
            display: true,
            text: 'Temperature (°C)'
          }
        }
      }
    }
  }
});

// Cleanup: destroy chart on component unmount
return () => {
  if (chartInstanceRef.current) {
    chartInstanceRef.current.destroy();
  }
};

}, [weatherData]));

const handleSearch = async (e) => {
  if (e) e.preventDefault();
  setLoading(true);
  setError('');

  try {
    const data = await fetchWeather(city);
    setWeatherData(data);
  } catch (err) {
    setError('City not found or network error. Please try again.');
```

Live Weather Visualization

```

    setWeatherData(null);
  } finally {
    setLoading(false);
  }
};

return (
  <div className="card shadow-sm p-4 bg-white mx-auto" style={{
    maxWidth: '700px' }}>
    <h3 className="text-center text-primary mb-4 fw-bold">Live
    Weather Visualization</h3>

    </* Input Form */>
    <form onSubmit={handleSearch} className="input-group mb-3">
      <input
        type="text"
        className="form-control"
        placeholder="Enter city name (e.g., London, Tokyo)"
        value={city}
        onChange={(e) => setCity(e.target.value)}
        required
      />
      <button type="submit" className="btn btn-primary"
disabled={loading}>
        {loading ? 'Fetching...' : 'Get Weather'}
      </button>
    </form>
  </div>
);

```

```

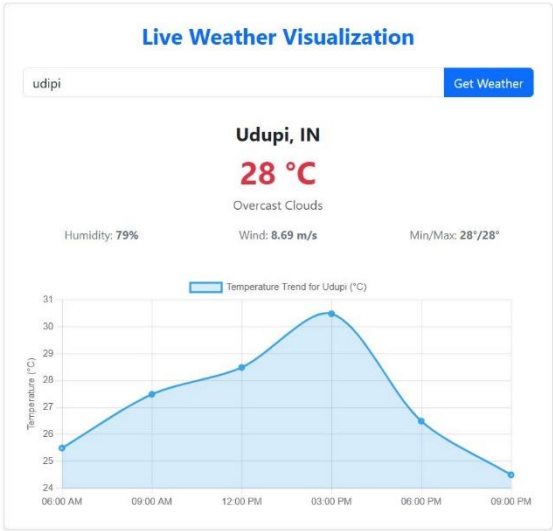
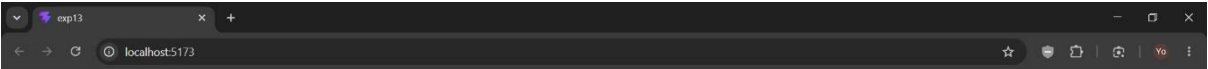
    { /* Error Alert Box */
    {error && <div className="alert alert-danger text-center">{error}</div>}

    { /* Weather Metrics display */
    {weatherData && (
        <div className="text-center my-3">
            <h4 className="fw-bold mb-1">{weatherData.name},
{weatherData.sys.country}</h4>
            <p className="fs-1 text-danger fw-bold mb-0">{Math.round(weatherData.main.temp)} &deg;C</p>
            <p className="text-muted text-capitalize mb-3">{weatherData.weather[0].description}</p>

            <div className="row text-center mb-4 text-secondary"
style={{ fontSize: '0.9rem' }}>
                <div className="col">Humidity:
<strong>{weatherData.main.humidity}%</strong></div>
                <div className="col">Wind:
<strong>{weatherData.wind.speed} m/s</strong></div>
                <div className="col">Min/Max:
<strong>{Math.round(weatherData.main.temp_min)} &deg;/{Math.round(weatherData.main.temp_max)} &deg;</strong></div>
            </div>
        </div>
    )}

    { /* Canvas Element for Chart.js Graph */
    <div className="chart-container" style={{ position: 'relative',
height: '300px', width: '100%' }}>
        <canvas ref={canvasRef}></canvas>
    </div>
</div>
    );
}

export default WeatherDashboard;
```



Experiment 14 Create a TODO application in react with necessary components and deploy it into GitHub.

structured into separated React components, along with step-by-step deployment instructions for GitHub Pages.

```
{
  "name": "react-todo-app",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "homepage": "https://praveenmunagapati.github.io/exp14",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "predeploy": "npm run build",
    "deploy": "gh-pages -d dist"
  },
  "dependencies": {
    "react": "^19.2.7",
    "react-dom": "^19.2.7"
  },
  "devDependencies": {
    "gh-pages": "^6.1.1",
    "@eslint/js": "^9.15.0",
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "@vitejs/plugin-react": "^4.3.4",
    "eslint": "^9.15.0",
    "eslint-plugin-react": "^7.37.2",
    "eslint-plugin-react-hooks": "^5.0.0",
    "eslint-plugin-react-refresh": "^0.4.14",
    "globals": "^15.12.0",
    "vite": "^6.0.0"
  }
}
```

```
<!-- ===== File Path: public/index.html ===== -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>React Todo Application</title>
  <!-- Bootstrap 5 CSS CDN -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body class="bg-light">
<div id="root"></div>
<script type="module" src="src/main.jsx"></script>
</body>
</html>
```

```
/* ===== File Path: src/main.jsx ===== */
```

```

import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);

/* ===== File Path: src/App.jsx ===== */
import React, { useState } from 'react';
import TodoForm from './components/TodoForm';
import TodoList from './components/TodoList';

function App() {
  const [todos, setTodos] = useState([
    { id: 1, text: "Explore Event modules in Node.js", completed: true },
    { id: 2, text: "Build a single page app in React", completed: false },
    { id: 3, text: "Verify local database connection using JDBC",
completed: false }
  ]);

  // Task Creation logic
  const addTodo = (text) => {
    const newTodo = {
      id: Date.now(), // Generates a unique timestamp ID
      text: text,
      completed: false
    };
    setTodos([...todos, newTodo]);
  };

  // Task state inversion logic (complete / incomplete)
  const toggleComplete = (id) => {
    setTodos(
      todos.map(todo =>
        todo.id === id ? { ...todo, completed: !todo.completed } : todo
      )
    );
  };

  // Task removal logic
  const deleteTodo = (id) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  return (
    <div className="container my-5">
      <div className="row">
        <div className="col-md-6 mx-auto">
          <div className="card p-4 shadow-sm bg-white">
            <h3 className="text-center mb-4 text-primary fw-bold">My Task
Tracker (Todo)</h3>

            <TodoForm addTodo={addTodo} />

            <TodoList
              todos={todos}
              toggleComplete={toggleComplete}
              deleteTodo={deleteTodo}
            />
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

        <div className="card-footer bg-white border-0 text-center mt-3 text-muted">
            <small>Completed: {todos.filter(t => t.completed).length} /
Total: {todos.length}</small>
        </div>
    </div>
</div>
</div>
</div>
</div>
    );
}

export default App;

```

```

/* ===== File Path: src/components/ToDoForm.jsx ===== */
import React, { useState } from 'react';

```

```

function ToDoForm({ addTodo }) {
    const [value, setValue] = useState('');

    const handleSubmit = (e) => {
        e.preventDefault();
        if (!value.trim()) return;
        addTodo(value); // Execute parent state addition trigger
        setValue(''); // Clear input box
    };

    return (
        <form onSubmit={handleSubmit} className="input-group mb-4">
            <input
                type="text"
                className="form-control"
                placeholder="Add a new task..."
                value={value}
                onChange={(e) => setValue(e.target.value)}
            />
            <button type="submit" className="btn btn-primary px-4">Add
Task</button>
        </form>
    );
}

export default ToDoForm;

```

```

/* ===== File Path: src/components/ToDoItem.jsx ===== */
import React from 'react';

```

```

function ToDoItem({ todo, toggleComplete, deleteTodo }) {
    return (
        <div className="list-group-item d-flex justify-content-between align-items-center py-3 my-1 border rounded bg-white">
            <div className="d-flex align-items-center flex-grow-1">
                <input
                    type="checkbox"
                    className="form-check-input me-3"
                    checked={todo.completed}
                    onChange={() => toggleComplete(todo.id)}
                    style={{ width: '20px', height: '20px', cursor:
'pointer' }}
                />
                <span

```

```

        style={{
          textDecoration: todo.completed ? 'line-through' :
'none',

          color: todo.completed ? '#6c757d' : '#212529',
          fontSize: '1.1rem'
        }}
      >
        {todo.text}
      </span>
    </div>
    <button
      className="btn btn-outline-danger btn-sm"
      onClick={() => deleteTodo(todo.id)}
    >
      Remove
    </button>
  </div>
);
}

export default TodoItem;

/* ===== File Path: src/components/ToDoList.jsx ===== */
import React from 'react';
import TodoItem from './TodoItem';

function ToDoList({ todos, toggleComplete, deleteTodo }) {
  if (todos.length === 0) {
    return (
      <div className="text-center text-muted my-4">
        <p className="fs-5">No tasks remaining. Keep it up!</p>
      </div>
    );
  }

  return (
    <div className="list-group">
      {todos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          toggleComplete={toggleComplete}
          deleteTodo={deleteTodo}
        />
      ))}
    </div>
  );
}

export default ToDoList;

```